

analysis

August 6, 2025

```
[1]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

```
[2]: games=pd.read_csv('games.csv')
games.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21925 entries, 0 to 21924
Data columns (total 48 columns):
#   Column                Non-Null Count  Dtype
---  -
0   BGGId                 21925 non-null  int64
1   Name                  21925 non-null  object
2   Description           21924 non-null  object
3   YearPublished         21925 non-null  int64
4   GameWeight            21925 non-null  float64
5   AvgRating             21925 non-null  float64
6   BayesAvgRating        21925 non-null  float64
7   StdDev                21925 non-null  float64
8   MinPlayers            21925 non-null  int64
9   MaxPlayers            21925 non-null  int64
10  ComAgeRec              16395 non-null  float64
11  LanguageEase           16034 non-null  float64
12  BestPlayers            21925 non-null  int64
13  GoodPlayers            21925 non-null  object
14  NumOwned               21925 non-null  int64
15  NumWant                21925 non-null  int64
16  NumWish                21925 non-null  int64
17  NumWeightVotes         21925 non-null  int64
18  MfgPlaytime            21925 non-null  int64
19  ComMinPlaytime         21925 non-null  int64
20  ComMaxPlaytime         21925 non-null  int64
21  MfgAgeRec              21925 non-null  int64
```

```

22 NumUserRatings      21925 non-null int64
23 NumComments         21925 non-null int64
24 NumAlternates       21925 non-null int64
25 NumExpansions       21925 non-null int64
26 NumImplementations  21925 non-null int64
27 IsReimplementation  21925 non-null int64
28 Family              6663 non-null object
29 Kickstarted         21925 non-null int64
30 ImagePath           21908 non-null object
31 Rank:boardgame      21925 non-null int64
32 Rank:strategygames  21925 non-null int64
33 Rank:abstracts      21925 non-null int64
34 Rank:familygames    21925 non-null int64
35 Rank:thematic       21925 non-null int64
36 Rank:cgs            21925 non-null int64
37 Rank:wargames       21925 non-null int64
38 Rank:partygames     21925 non-null int64
39 Rank:childrensgames 21925 non-null int64
40 Cat:Thematic        21925 non-null int64
41 Cat:Strategy        21925 non-null int64
42 Cat:War             21925 non-null int64
43 Cat:Family          21925 non-null int64
44 Cat:CGS             21925 non-null int64
45 Cat:Abstract        21925 non-null int64
46 Cat:Party           21925 non-null int64
47 Cat:Childrens       21925 non-null int64
dtypes: float64(6), int64(37), object(5)
memory usage: 8.0+ MB

```

```
[3]: games.columns
```

```

[3]: Index(['BGGId', 'Name', 'Description', 'YearPublished', 'GameWeight',
        'AvgRating', 'BayesAvgRating', 'StdDev', 'MinPlayers', 'MaxPlayers',
        'ComAgeRec', 'LanguageEase', 'BestPlayers', 'GoodPlayers', 'NumOwned',
        'NumWant', 'NumWish', 'NumWeightVotes', 'MfgPlaytime', 'ComMinPlaytime',
        'ComMaxPlaytime', 'MfgAgeRec', 'NumUserRatings', 'NumComments',
        'NumAlternates', 'NumExpansions', 'NumImplementations',
        'IsReimplementation', 'Family', 'Kickstarted', 'ImagePath',
        'Rank:boardgame', 'Rank:strategygames', 'Rank:abstracts',
        'Rank:familygames', 'Rank:thematic', 'Rank:cgs', 'Rank:wargames',
        'Rank:partygames', 'Rank:childrensgames', 'Cat:Thematic',
        'Cat:Strategy', 'Cat:War', 'Cat:Family', 'Cat:CGS', 'Cat:Abstract',
        'Cat:Party', 'Cat:Childrens'],
        dtype='object')

```

```
[4]: game_cols=['BGId', 'Name', 'YearPublished', 'MfgPlaytime',
               'MinPlayers', 'MaxPlayers', 'MfgAgeRec',
               'NumUserRatings', 'NumOwned', 'NumWant', 'NumWish']
games=games[game_cols]
games.to_csv('games_reduced.csv')
```

```
[5]: games.head()
```

```
[5]:
```

	BGId	Name	YearPublished	MfgPlaytime	MinPlayers	MaxPlayers	\
0	1	Die Macher	1986	240	3	5	
1	2	Dragonmaster	1981	30	3	4	
2	3	Samurai	1998	60	2	4	
3	4	Tal der Könige	1992	60	2	4	
4	5	Acquire	1964	90	2	6	

	MfgAgeRec	NumUserRatings	NumOwned	NumWant	NumWish
0	14	5354	7498	501	2039
1	12	562	1285	72	191
2	10	15146	15578	799	3450
3	12	340	638	54	123
4	12	18655	23735	548	2671

```
[6]: games.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21925 entries, 0 to 21924
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   BGId                  21925 non-null  int64
1   Name                  21925 non-null  object
2   YearPublished         21925 non-null  int64
3   MfgPlaytime           21925 non-null  int64
4   MinPlayers            21925 non-null  int64
5   MaxPlayers            21925 non-null  int64
6   MfgAgeRec             21925 non-null  int64
7   NumUserRatings        21925 non-null  int64
8   NumOwned              21925 non-null  int64
9   NumWant               21925 non-null  int64
10  NumWish               21925 non-null  int64
dtypes: int64(10), object(1)
memory usage: 1.8+ MB
```

```
[7]: games.describe()
```

```
[7]:
```

	BGId	YearPublished	MfgPlaytime	MinPlayers	MaxPlayers	\
count	21925.000000	21925.000000	21925.000000	21925.000000	21925.000000	
mean	117652.663216	1985.494914	90.513523	2.007343	5.707868	

std	104628.721777	212.486214	529.657389	0.693093	15.014643
min	1.000000	-3500.000000	0.000000	0.000000	0.000000
25%	12346.000000	2001.000000	25.000000	2.000000	4.000000
50%	105305.000000	2011.000000	45.000000	2.000000	4.000000
75%	206169.000000	2017.000000	90.000000	2.000000	6.000000
max	349161.000000	2021.000000	60000.000000	10.000000	999.000000

	MfgAgeRec	NumUserRatings	NumOwned	NumWant	NumWish
count	21925.000000	21925.000000	21925.000000	21925.000000	21925.000000
mean	9.613409	861.668324	1467.848164	41.690946	228.457013
std	3.641560	3638.680857	5294.120574	117.255229	788.477151
min	0.000000	30.000000	0.000000	0.000000	0.000000
25%	8.000000	56.000000	150.000000	3.000000	14.000000
50%	10.000000	123.000000	320.000000	9.000000	39.000000
75%	12.000000	395.000000	899.000000	28.000000	127.000000
max	25.000000	108101.000000	166497.000000	2031.000000	19182.000000

```
[8]: games[games['YearPublished']<0][['Name', 'YearPublished']]
```

```
[8]:
```

	Name	YearPublished
164	Go	-2200
1195	The Royal Game of Ur	-2600
1721	Backgammon	-3000
1723	Senet	-3500
2574	Nine Men's Morris	-1400
3304	Chutes and Ladders	-200
3356	Marbles	-3000
5345	Knucklebones	-3000
5378	Tic-Tac-Toe	-1300
6640	Three Men's Morris	-1400
6858	Petteia	-100

```
[9]: games[games['YearPublished']==0][['Name', 'YearPublished']].head()
```

```
[9]:
```

	Name	YearPublished
2052	Lepanto	0
2646	Jungle	0
2745	Mao	0
3009	Sinaí	0
3146	Carrom	0

```
[10]: games[games['YearPublished']==0]['Name'].size
```

```
[10]: 193
```

```
[11]: for column in games.columns:
        number_zeros=games[games[column]==0]['Name'].size
        if number_zeros>0:
            print(f"{column} has 0 {number_zeros} times")
```

```
YearPublished has 0 193 times
MfgPlaytime has 0 780 times
MinPlayers has 0 50 times
MaxPlayers has 0 173 times
MfgAgeRec has 0 1325 times
NumOwned has 0 1 times
NumWant has 0 1938 times
NumWish has 0 106 times
```

```
[12]: games[games['MfgAgeRec']==0]['Name'].head()
```

```
[12]: 145      Black Death
      248      Rossyia 1917
      323      Wrott & Swindlers
      387      Beutelschneider
      478      Doolittle & Waite
      Name: Name, dtype: object
```

```
[13]: replaceZeroList=games.columns[2: -4]
        for column in replaceZeroList:
            median=games[column].median()
            games[column].replace(0, median,inplace=True)
            print(f"median {column} is {median}")
```

```
median YearPublished is 2011.0
median MfgPlaytime is 45.0
median MinPlayers is 2.0
median MaxPlayers is 4.0
median MfgAgeRec is 10.0
```

C:\Users\op101\AppData\Local\Temp\ipykernel_13768\112971529.py:4: FutureWarning:
A value is trying to be set on a copy of a DataFrame or Series through chained
assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behaves as
a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using
'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value)
instead, to perform the operation inplace on the original object.

```
games[column].replace(0, median,inplace=True)
```

C:\Users\op101\AppData\Local\Temp\ipykernel_13768\112971529.py:4: FutureWarning:
A value is trying to be set on a copy of a DataFrame or Series through chained
assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behaves as
a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using
'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value)
instead, to perform the operation inplace on the original object.

```
games[column].replace(0, median,inplace=True)
```

C:\Users\op101\AppData\Local\Temp\ipykernel_13768\112971529.py:4: FutureWarning:
A value is trying to be set on a copy of a DataFrame or Series through chained
assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behaves as
a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using
'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value)
instead, to perform the operation inplace on the original object.

```
games[column].replace(0, median,inplace=True)
```

C:\Users\op101\AppData\Local\Temp\ipykernel_13768\112971529.py:4: FutureWarning:
A value is trying to be set on a copy of a DataFrame or Series through chained
assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behaves as
a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using
'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value)
instead, to perform the operation inplace on the original object.

```
games[column].replace(0, median,inplace=True)
```

C:\Users\op101\AppData\Local\Temp\ipykernel_13768\112971529.py:4: FutureWarning:
A value is trying to be set on a copy of a DataFrame or Series through chained
assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behaves as
a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using

'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value)
instead, to perform the operation inplace on the original object.

```
games[column].replace(0, median,inplace=True)
```

```
[14]: for column in games.columns:
        number_zeros=games[games[column]==0]['Name'].size
        if number_zeros>0:
            print(f"{column} has 0 {number_zeros} times")
```

```
NumOwned has 0 1 times
NumWant has 0 1938 times
NumWish has 0 106 times
```

```
[15]: games.to_csv("games_filled.csv")
```

```
[16]: df_mechanics=pd.read_csv('mechanics.csv')
df_mechanics.head()
```

```
[16]:
```

	BGGId	Alliances	Area Majority / Influence	Auction/Bidding	Dice Rolling	\
0	1	1	1	1	1	
1	2	0	0	0	0	
2	3	0	1	0	0	
3	4	0	1	1	0	
4	5	0	0	0	0	

	Hand Management	Simultaneous Action Selection	Trick-taking	Hexagon Grid	\
0	1	1	0	0	
1	0	0	1	0	
2	1	0	0	1	
3	0	0	0	0	
4	1	0	0	0	

	Once-Per-Game Abilities	...	Contracts	Passed Action Token	\
0	0	...	0	0	
1	0	...	0	0	
2	1	...	0	0	
3	0	...	0	0	
4	0	...	0	0	

	King of the Hill	Action Retrieval	Force Commitment	Rondel	\
0	0	0	0	0	
1	0	0	0	0	
2	0	0	0	0	

3	0	0	0	0
4	0	0	0	0

	Automatic Resource Growth	Legacy Game	Dexterity	Physical
0	0	0	0	0
1	0	0	0	0
2	0	0	0	0
3	0	0	0	0
4	0	0	0	0

[5 rows x 158 columns]

```
[17]: df_mechanics=pd.read_csv('mechanics.csv')
df_mechanics.head()
```

```
[17]:
```

	BGGId	Alliances	Area Majority / Influence	Auction/Bidding	Dice Rolling	\
0	1	1		1	1	1
1	2	0		0	0	0
2	3	0		1	0	0
3	4	0		1	1	0
4	5	0		0	0	0

	Hand Management	Simultaneous Action Selection	Trick-taking	Hexagon Grid	\
0	1		1	0	0
1	0		0	1	0
2	1		0	0	1
3	0		0	0	0
4	1		0	0	0

	Once-Per-Game Abilities	...	Contracts	Passed Action Token	\
0	0	...	0	0	
1	0	...	0	0	
2	1	...	0	0	
3	0	...	0	0	
4	0	...	0	0	

	King of the Hill	Action Retrieval	Force Commitment	Rondel	\
0	0	0	0	0	
1	0	0	0	0	
2	0	0	0	0	
3	0	0	0	0	
4	0	0	0	0	

	Automatic Resource Growth	Legacy Game	Dexterity	Physical
0	0	0	0	0
1	0	0	0	0
2	0	0	0	0

3	0	0	0	0
4	0	0	0	0

[5 rows x 158 columns]

```
[18]: df_mechanics=pd.read_csv('Table_mechanics.csv')
df_mechanics.head()
```

```
[18]: Unnamed: 0      1      2      3 \
0      0 Alliances Area Majority / Influence Auction/Bidding
1      1 Alliances Area Majority / Influence Auction/Bidding
2      2 Alliances Area Majority / Influence Auction/Bidding
3      3 Alliances Area Majority / Influence Auction/Bidding
4      4 Alliances Area Majority / Influence Auction/Bidding

      4      5      6      7 \
0 Dice Rolling Hand Management Simultaneous Action Selection Trick-taking
1 Dice Rolling Hand Management Simultaneous Action Selection Trick-taking
2 Dice Rolling Hand Management Simultaneous Action Selection Trick-taking
3 Dice Rolling Hand Management Simultaneous Action Selection Trick-taking
4 Dice Rolling Hand Management Simultaneous Action Selection Trick-taking

      8      9 ...      148      149 \
0 Hexagon Grid Once-Per-Game Abilities ... Contracts Passed Action Token
1 Hexagon Grid Once-Per-Game Abilities ... Contracts Passed Action Token
2 Hexagon Grid Once-Per-Game Abilities ... Contracts Passed Action Token
3 Hexagon Grid Once-Per-Game Abilities ... Contracts Passed Action Token
4 Hexagon Grid Once-Per-Game Abilities ... Contracts Passed Action Token

      150      151      152      153 \
0 King of the Hill Action Retrieval Force Commitment Rondel
1 King of the Hill Action Retrieval Force Commitment Rondel
2 King of the Hill Action Retrieval Force Commitment Rondel
3 King of the Hill Action Retrieval Force Commitment Rondel
4 King of the Hill Action Retrieval Force Commitment Rondel

      154      155      156      157
0 Automatic Resource Growth Legacy Game Dexterity Physical
1 Automatic Resource Growth Legacy Game Dexterity Physical
2 Automatic Resource Growth Legacy Game Dexterity Physical
3 Automatic Resource Growth Legacy Game Dexterity Physical
4 Automatic Resource Growth Legacy Game Dexterity Physical
```

[5 rows x 158 columns]

```
[19]: df_mechanics.iloc[:,1:].value_counts()
```

[19]: 1 2 3 4 5
6 7 8 9
10 11 12 13 14 15
16 17 18 19 20
21 22 23 24
25 26 27 28 29 30 31
32 33 34 35 36
37 38 39 40 41
42 43 44 45 46
47 48 49 50 51
52 53 54 55
56 57 58 59 60 61
62 63 64 65
66 67 68 69
70 71 72 73 74
75 76 77 78 79 80
81 82 83 84 85
86 87 88 89 90
91 92 93 94 95 96
97 98 99 100 101 102
103 104 105 106 107 108
109 110 111 112 113
114 115 116 117 118
119 120 121 122
123 124 125 126
127 128 129 130
131 132 133 134 135 136
137 138 139 140
141 142 143 144 145
146 147 148 149 150
151 152 153 154 155
156 157

Alliances Area Majority / Influence Auction/Bidding Dice Rolling Hand
Management Simultaneous Action Selection Trick-taking Hexagon Grid Once-Per-
Game Abilities Set Collection Tile Placement Action Points Investment
Market Square Grid Stock Holding Victory Points as a Resource Enclosure
Pattern Building Pattern Recognition Modular Board Network and Route Building
Point to Point Movement Melding and Splaying Negotiation Trading Push Your
Luck Income Race Random Production Variable Set-up Roll / Spin and Move
Variable Player Powers Action Queue Bias Grid Movement Lose a Turn
Programmed Movement Scenario / Mission / Campaign Game Voting Events Paper-
and-Pencil Player Elimination Role Playing Movement Points Simulation
Variable Phase Order Area Movement Commodity Speculation Cooperative Game
Deduction Sudden Death Ending Connections Highest-Lowest Scoring Betting and
Bluffing Memory Score-and-Reset Game Layering Map Addition Secret Unit
Deployment Increase Value of Unchosen Resources Ratio / Combat Results Table
Take That Team-Based Game Campaign / Battle Card Driven Tech Trees / Tech

Tracks Player Judge Chit-Pull System Three Dimensional Movement Action
 Drafting Minimap Resolution Stat Check Resolution Action Timer Pick-up and
 Deliver Map Deformation Bingo Crayon Rail System Multiple Maps Hidden Roles
 Line Drawing Tug of War Pattern Movement Static Capture Different Dice
 Movement Chaining Ladder Climbing Predictive Bid Solo / Solitaire Game Line
 of Sight Critical Hits and Failures Interrupts Zone of Control Bribery End
 Game Bonuses Area-Impulse Worker Placement Measurement Movement Map
 Reduction Real-Time Resource to Move Mancala Ownership Kill Steal Hidden
 Movement Track Movement Deck Construction Drafting TableauBuilding
 Prisoner's Dilemma Hidden Victory Points Movement Template Slide/Push
 Targeted Clues Command Cards Grid Coverage Relative Movement Action/Event
 Card Play Conflict Resolution I Cut, You Choose Die Icon Resolution Elapsed
 Real Time Ending Advantage Token Storytelling Catch the Leader Roles with
 Asymmetric Information Traitor Game Moving Multiple Units Semi-Cooperative
 Game Communication Limits Time Track Speed Matching Cube Tower Re-rolling
 and Locking Impulse Movement Loans Delayed Purchase Deck, Bag, and Pool
 Building Move Through Deck Single Loser Game Matching Induction Physical
 Removal Narrative Choice / Paragraph Pieces as Map Follow Finale Ending
 Order Counters Contracts Passed Action Token King of the Hill Action
 Retrieval Force Commitment Rondel Automatic Resource Growth Legacy Game
 Dexterity Physical 157
 Name: count, dtype: int64

```
[20]: df_mechanics.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 157 entries, 0 to 156
Columns: 158 entries, Unnamed: 0 to 157
dtypes: int64(1), object(157)
memory usage: 193.9+ KB
  
```

```
[21]: games_reduces=pd.read_csv("games_reduced.csv")
      games_reduces.head()
```

```

[21]: Unnamed: 0  BGGId      Name  YearPublished  MfgPlaytime  MinPlayers  \
0           0      1    Die Macher      1986           240           3
1           1      2  Dragonmaster      1981           30           3
2           2      3    Samurai      1998           60           2
3           3      4  Tal der Könige      1992           60           2
4           4      5    Acquire      1964           90           2

      MaxPlayers  MfgAgeRec  NumUserRatings  NumOwned  NumWant  NumWish
0             5          14           5354      7498      501      2039
1             4          12            562      1285       72       191
2             4          10          15146     15578      799     3450
3             4          12            340       638       54       123
4             6          12          18655     23735     548     2671
  
```

```
[22]: datasets={
    "artist":pd.read_csv("artists_reduced.csv"),
    "designer":pd.read_csv('designers_reduced.csv'),
    "mechanic":pd.read_csv("mechanics.csv"),
    "publisher":pd.read_csv("publishers_reduced.csv"),
    "subcategory":pd.read_csv("subcategories.csv"),
    "themes":pd.read_csv("themes.csv")
}
```

```
[23]: datasets["artist"].head()
```

```
[23]:
```

	Harald Lieske	Franz Vohwinkel	Peter Whitley	Scott Okumura	(Uncredited)	\
0	1	0	0	0	0	
1	0	0	0	0	0	
2	0	1	0	0	0	
3	0	0	0	0	0	
4	0	0	1	1	0	

	Doris Matthäus	Alan R. Moon	Alexander Jung	Andrea Boekhoff	\
0	0	0	0	0	
1	0	0	0	0	
2	0	0	0	0	
3	0	0	0	0	
4	0	0	0	0	

	Björn Pertoft	...	Nathan Meunier	Andrey Gordeev	Zbigniew Umgelter	\
0	0	...	0	0	0	
1	0	...	0	0	0	
2	0	...	0	0	0	
3	0	...	0	0	0	
4	0	...	0	0	0	

	Jeppe Norsker	Daniel Profiri	Aleksander Zawada	Simon Douchy	\
0	0	0	0	0	
1	0	0	0	0	
2	0	0	0	0	
3	0	0	0	0	
4	0	0	0	0	

	Felix Wermke	BGGId	Low-Exp	Artist
0	0	1		1
1	0	2		1
2	0	3		0
3	0	4		1
4	0	5		0

```
[5 rows x 1681 columns]
```

()

```
[24]: for dataset in datasets.keys():
      df=datasets[dataset]
      if 'BGGId' in df.columns:
          datasets[dataset] = pd.concat([df[['BGGId']], df.
↳drop(columns=['BGGId'])], axis=1)
```

```
[25]: datasets['artist'].head()
```

```
[25]: BGGId  Harald Lieske  Franz Vohwinkel  Peter Whitley  Scott Okumura  \
0         1              1              0              0              0
1         2              0              0              0              0
2         3              0              1              0              0
3         4              0              0              0              0
4         5              0              0              1              1
```

```
(Uncredited) Doris Matthäus  Alan R. Moon  Alexander Jung  \
0              0              0              0
1              0              0              0
2              0              0              0
3              0              0              0
4              0              0              0
```

```
Andrea Boekhoff  ...  Oliver Barrett  Nathan Meunier  Andrey Gordeev  \
0              0  ...              0              0              0
1              0  ...              0              0              0
2              0  ...              0              0              0
3              0  ...              0              0              0
4              0  ...              0              0              0
```

```
Zbigniew Umgelter  Jeppe Norsker  Daniel Profiri  Aleksander Zawada  \
0              0              0              0              0
1              0              0              0              0
2              0              0              0              0
3              0              0              0              0
4              0              0              0              0
```

```
Simon Douchy  Felix Wermke  Low-Exp Artist
0              0              0              1
1              0              0              1
2              0              0              0
3              0              0              1
4              0              0              0
```

[5 rows x 1681 columns]

/ /

```

[26]: for name, df in datasets.items():
    print(f"=== Dataset: {name} ===")

    bin_cols = df.columns[1:]
    data_bin = df[bin_cols]

    # - , == 0
    zero_cols = (data_bin.sum(axis=0) == 0).sum()
    print(f" : {zero_cols}")

    # - , == 0
    zero_rows = (data_bin.sum(axis=1) == 0).sum()
    print(f" : {zero_rows}")

    # -
    tags_per_game = data_bin.sum(axis=1)
    mean_tags_per_game = tags_per_game.mean()
    print(f" : {mean_tags_per_game:.2f}")

    # -
    games_per_tag = data_bin.sum(axis=0)
    mean_games_per_tag = games_per_tag.mean()
    print(f" : {mean_games_per_tag:.2f}")

    # -----
    fig, axs = plt.subplots(1, 2, figsize=(12,4))
    fig.suptitle(f" {name}")

    # :
    axs[0].hist(tags_per_game, bins=30, color='skyblue')
    axs[0].set_title(" - ")
    axs[0].set_xlabel(" ")
    axs[0].set_ylabel(" ")

    # :
    axs[1].hist(games_per_tag, bins=30, color='salmon')
    axs[1].set_title(" - ")
    axs[1].set_xlabel(" ")
    axs[1].set_ylabel(" ")

    plt.tight_layout()
    plt.show()

    print()

```

```

=== Dataset: artist ===
      : 0
      : 5997

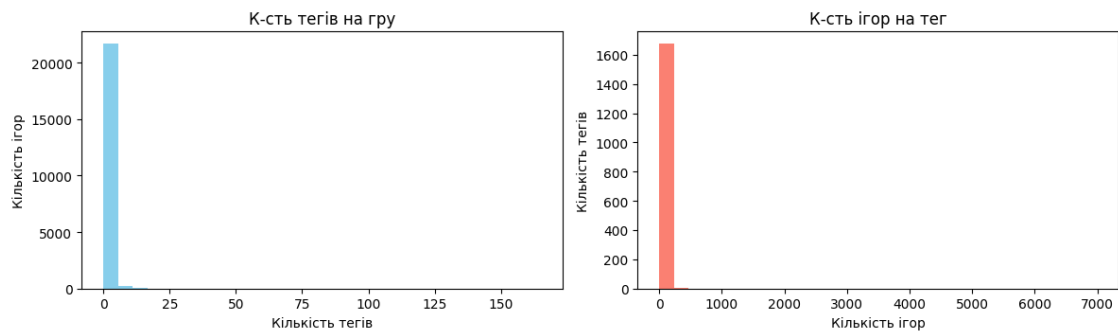
```

```

: 1.19
: 15.58

```

Розподіли для artist



```

=== Dataset: designer ===

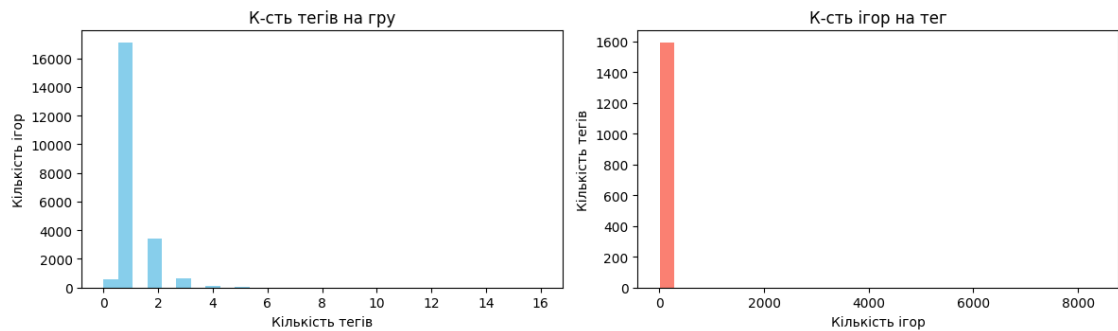
```

```

: 0
: 599
: 1.22
: 16.79

```

Розподіли для designer



```

=== Dataset: mechanic ===

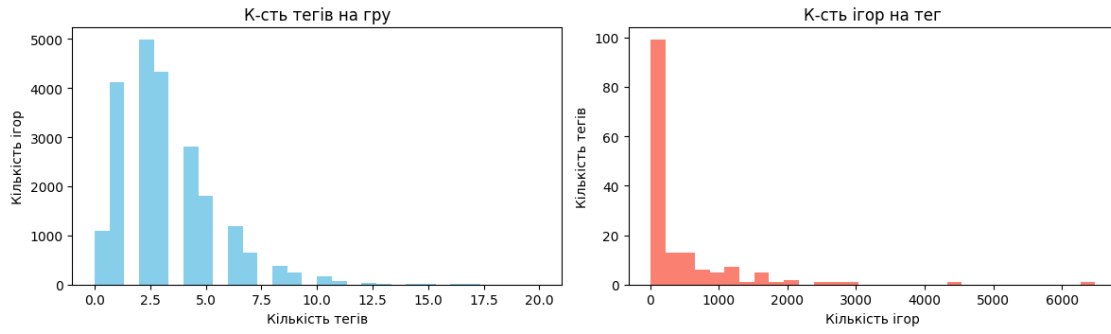
```

```

: 0
: 1084
: 3.11
: 433.63

```

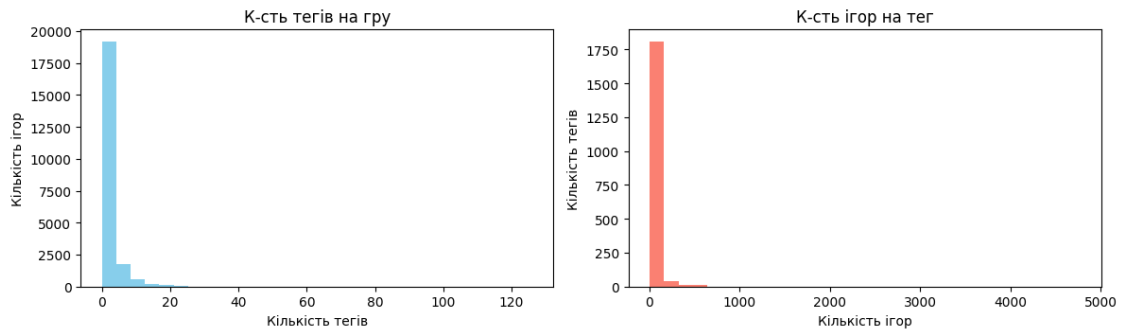
Розподіли для mechanic



=== Dataset: publisher ===

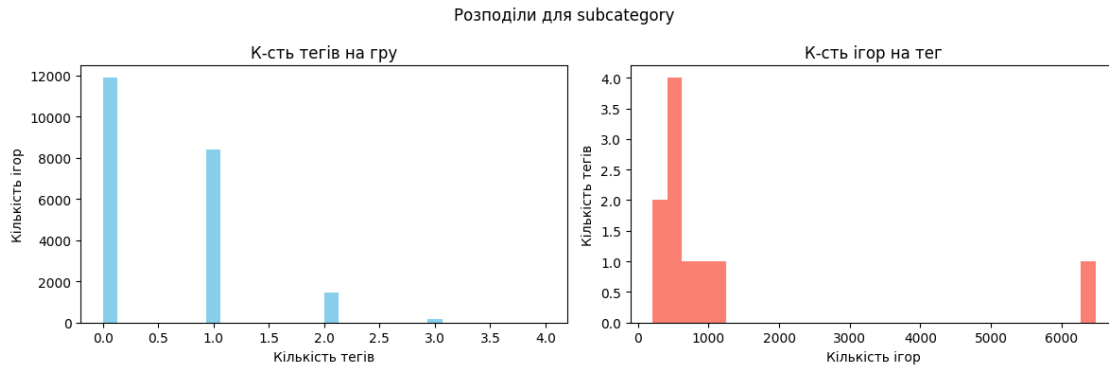
: 0
: 1
: 2.58
: 30.35

Розподіли для publisher

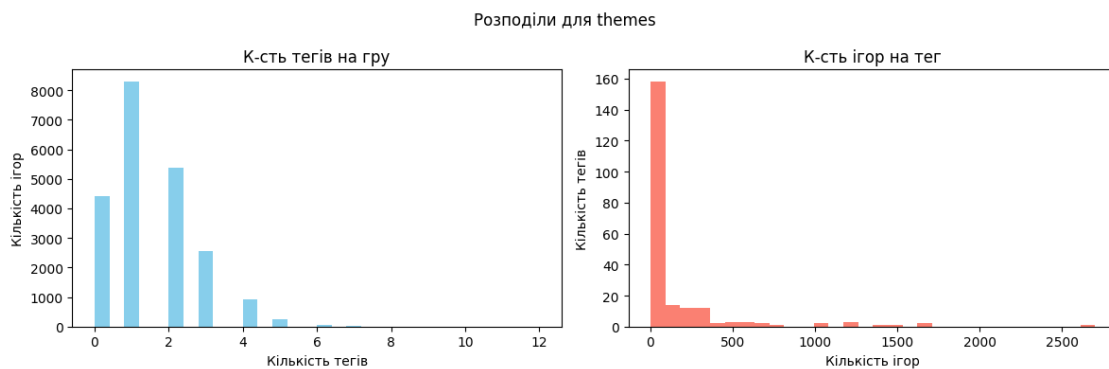


=== Dataset: subcategory ===

: 0
: 11892
: 0.54
: 1181.00



```
=== Dataset: themes ===
      : 0
      : 4410
      : 1.48
      : 149.21
```



```
[27]: ratings=pd.read_csv('ratings_distribution.csv')
      ratings.head()
```

```
[27]:  BGGId  0.0  0.1  0.5  1.0  1.1  1.2  1.3  1.4  1.5  ...  9.2  9.3  9.4  \
0      1  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  ... 11.0  5.0 11.0
1      2  0.0  0.0  0.0  3.0  0.0  0.0  0.0  0.0  0.0  ...  0.0  1.0  0.0
2      3  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  ... 20.0  7.0  4.0
3      4  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  ...  0.0  0.0  0.0
4      5  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  ... 18.0  8.0  4.0
```

	9.5	9.6	9.7	9.8	9.9	10.0	total_ratings
0	86.0	3.0	4.0	6.0	8.0	426.0	5352.0
1	1.0	0.0	0.0	1.0	0.0	17.0	562.0
2	77.0	3.0	1.0	5.0	3.0	477.0	15148.0
3	0.0	0.0	0.0	0.0	0.0	3.0	342.0
4	82.0	7.0	9.0	10.0	5.0	905.0	18387.0

[5 rows x 96 columns]

```
[28]: ratings.columns
```

```
[28]: Index(['BGGId', '0.0', '0.1', '0.5', '1.0', '1.1', '1.2', '1.3', '1.4', '1.5',
            '1.6', '1.7', '1.8', '1.9', '2.0', '2.1', '2.2', '2.3', '2.4', '2.5',
            '2.6', '2.7', '2.8', '2.9', '3.0', '3.1', '3.2', '3.3', '3.4', '3.5',
            '3.6', '3.7', '3.8', '3.9', '4.0', '4.1', '4.2', '4.3', '4.4', '4.5',
            '4.6', '4.7', '4.8', '4.9', '5.0', '5.1', '5.2', '5.3', '5.4', '5.5',
            '5.6', '5.7', '5.8', '5.9', '6.0', '6.1', '6.2', '6.3', '6.4', '6.5',
            '6.6', '6.7', '6.8', '6.9', '7.0', '7.1', '7.2', '7.3', '7.4', '7.5',
            '7.6', '7.7', '7.8', '7.9', '8.0', '8.1', '8.2', '8.3', '8.4', '8.5',
            '8.6', '8.7', '8.8', '8.9', '9.0', '9.1', '9.2', '9.3', '9.4', '9.5',
            '9.6', '9.7', '9.8', '9.9', '10.0', 'total_ratings'],
            dtype='object')
```

```
[29]: #
rating_cols = [col for col in ratings.columns if col not in ['BGGId',
↳ 'total_ratings']]

#
bin_map = {}
for col in rating_cols:
    val = float(col)
    if val < 9.0:
        bin_idx = int(val)
    else:
        bin_idx = 9
    bin_map.setdefault(bin_idx, []).append(col)

rating_values = [float(col) for col in rating_cols]

#
weighted_sum = (
    ratings[rating_cols] * rating_values
).sum(axis=1)
```

```

ratings['average_rating'] = weighted_sum / ratings['total_ratings']

bin_labels = []
for v in rating_values:
    if v >= 9.0:
        bin_labels.append(9)
    else:
        bin_labels.append(int(v))

#
bin_mapping_df = pd.DataFrame({
    'col': rating_cols,
    'bin': bin_labels
})

#
bins = bin_mapping_df.groupby('bin')['col'].apply(list)

#
ratings_bins = pd.DataFrame()
for bin_num, cols in bins.items():
    ratings_bins[f'{bin_num}'] = ratings[cols].sum(axis=1)

#
result = pd.concat([ratings['BGGIId'], ratings['average_rating'],
    ratings['total_ratings'], ratings_bins], axis=1)

result.to_csv("ratings_binned.csv")

result.head()

```

```

[29]:
  BGGIId  average_rating  total_ratings  0  1  2  3  4  5  \
0      1      7.672104      5352.0  0.0  0.0  24.0  60.0  92.0  231.0
1      2      6.642883       562.0  0.0  3.0   2.0   9.0  19.0   75.0
2      3      7.484335     15148.0  0.0  0.0   0.0  16.0 170.0  506.0
3      4      6.599415       342.0  0.0  0.0   0.0   5.0  10.0   44.0
4      5      7.439860     18387.0  0.0  0.0   0.0   0.0  56.0  945.0

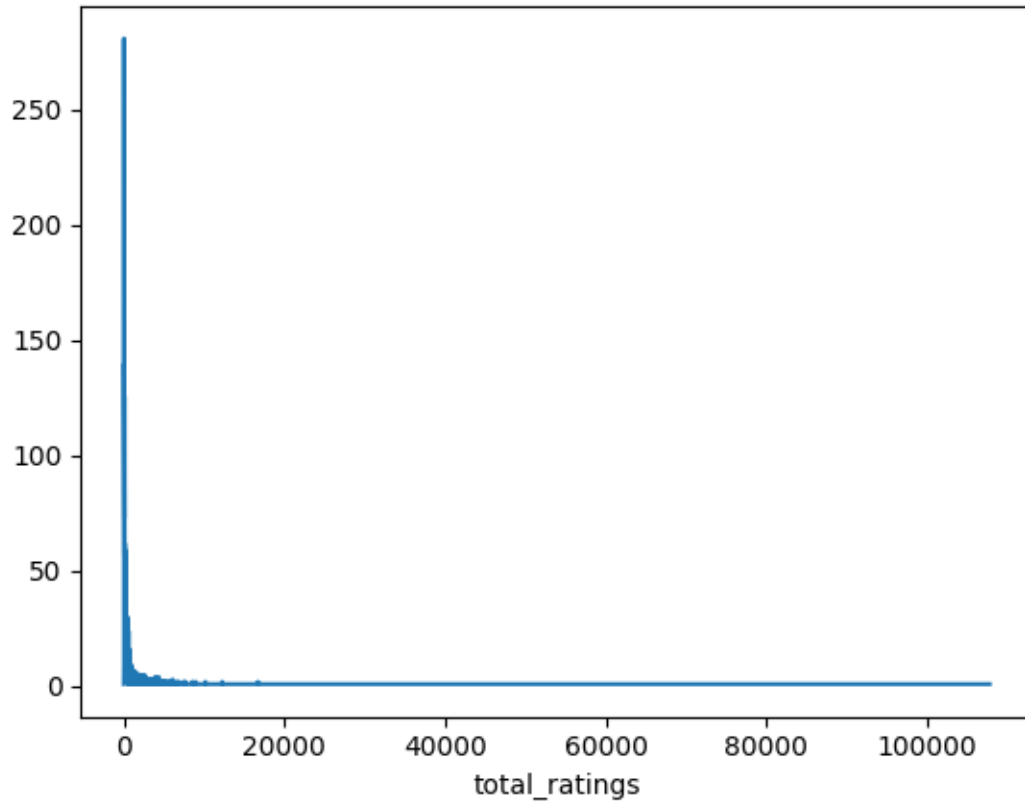
      6      7      8      9
0  628.0 1312.0 1598.0 1407.0
1  162.0  157.0   83.0   52.0
2 2039.0 5505.0 4807.0 2105.0
3  109.0  107.0   49.0   18.0
4 2996.0 6587.0 4952.0 2851.0

```

```
[30]: games_regression=pd.concat([games, ratings['average_rating']],axis=1)
      games_classification=pd.concat([games, ratings['average_rating'].apply(lambda x:
      ↪ int(x))],axis=1)
```

```
[31]: ratings['total_ratings'].value_counts().sort_index().plot()
```

```
[31]: <Axes: xlabel='total_ratings'>
```



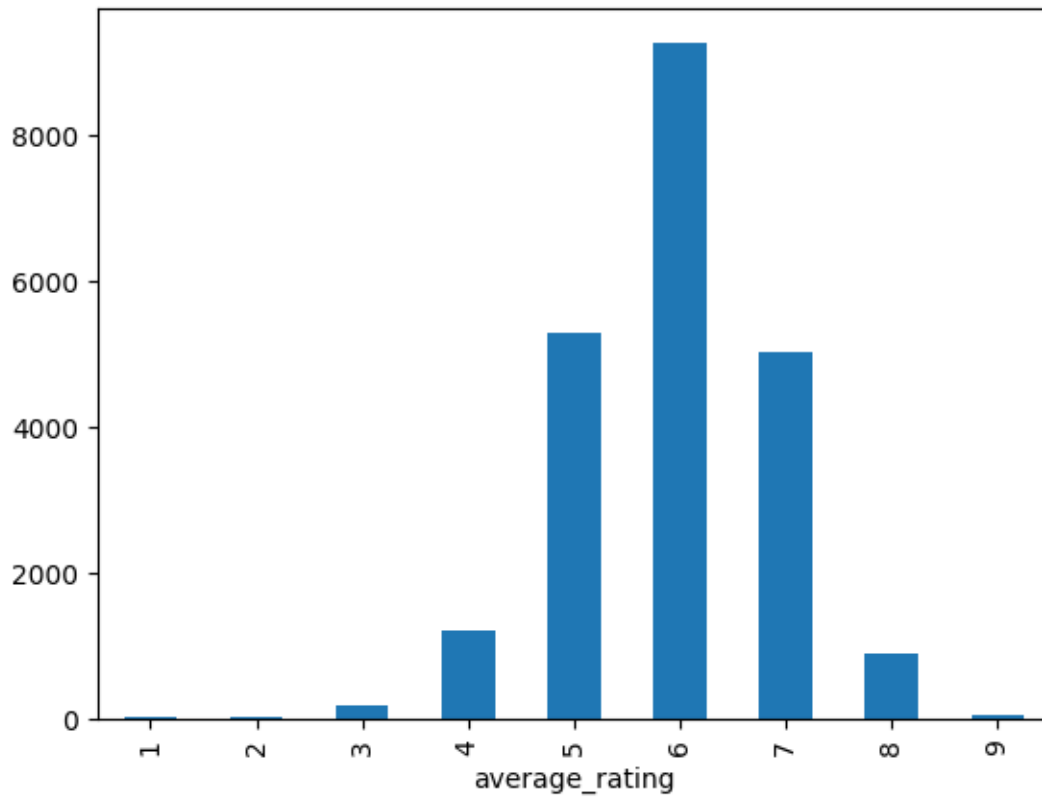
```
[32]: games_classification.average_rating.value_counts().sort_index()
```

```
[32]: average_rating
      1      8
      2     25
      3    177
      4   1216
      5   5276
      6   9266
      7  5023
```

```
8      897
9       37
Name: count, dtype: int64
```

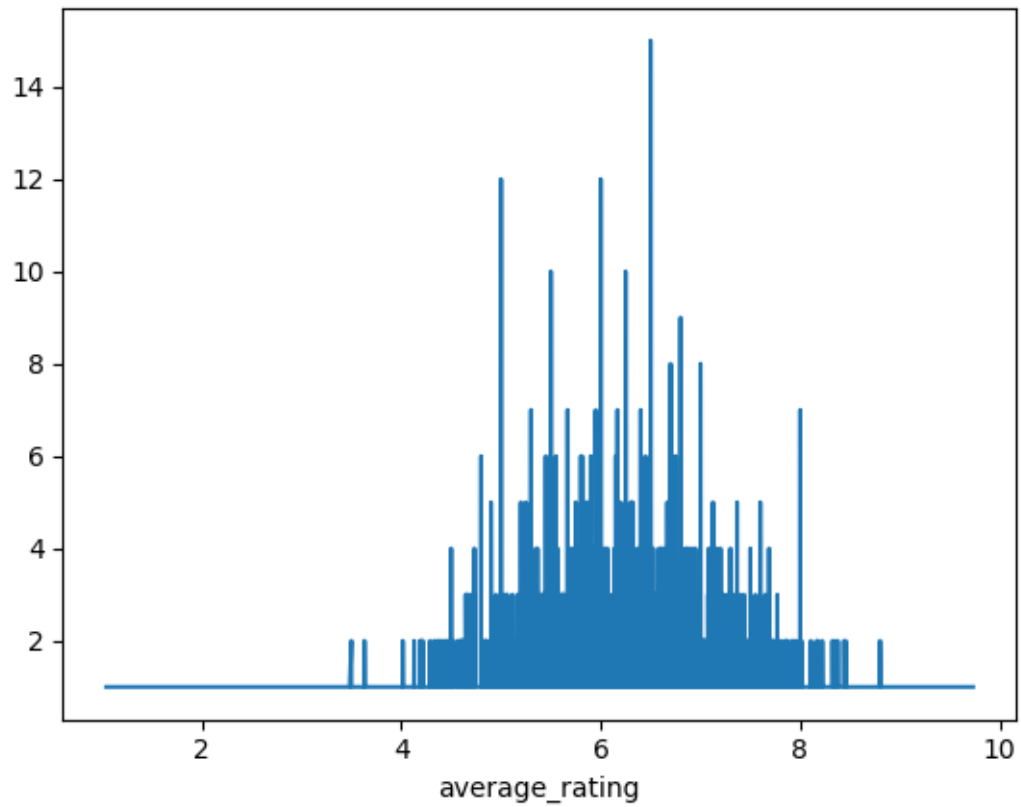
```
[33]: games_classification.average_rating.value_counts().sort_index().plot(kind="bar")
```

```
[33]: <Axes: xlabel='average_rating'>
```

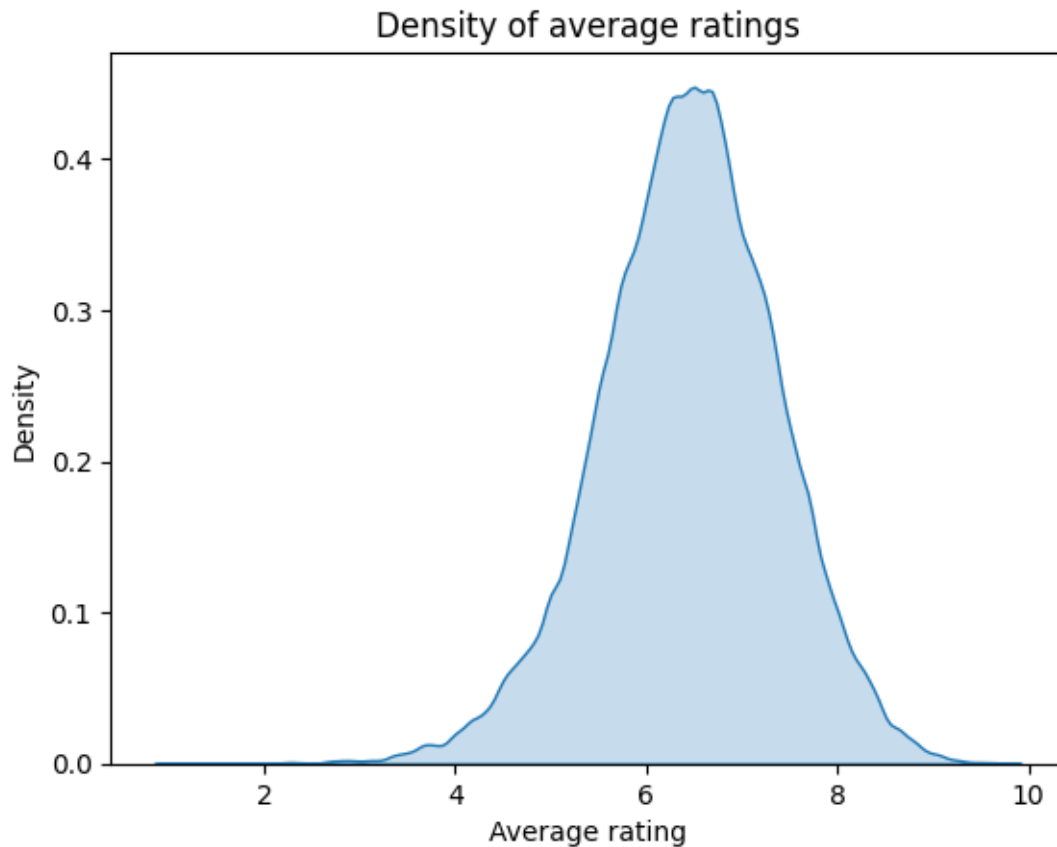


```
[34]: games_regression.average_rating.value_counts().sort_index().plot()
```

```
[34]: <Axes: xlabel='average_rating'>
```



```
[35]: sns.kdeplot(games_regression.average_rating, fill=True, bw_adjust=0.5)
plt.xlabel("Average rating")
plt.title("Density of average ratings")
plt.show()
```

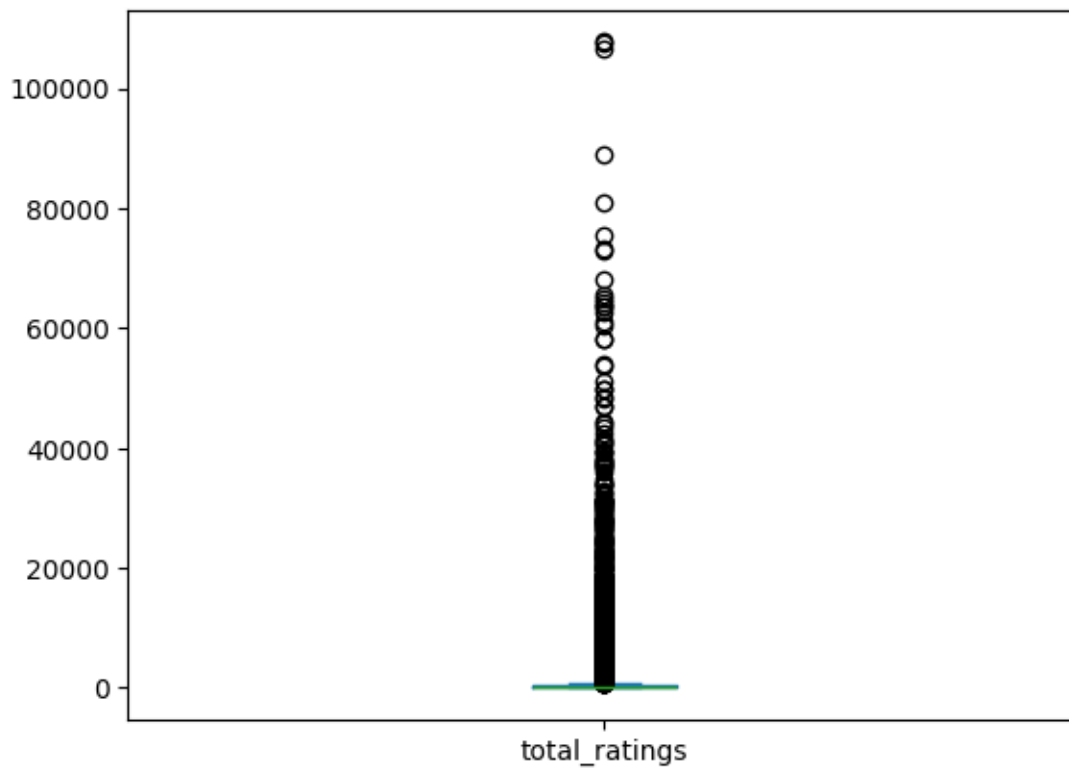


```
[36]: from sklearn.model_selection import train_test_split
class_df_train, class_df_test, class_labels_train, \
    class_labels_test=train_test_split(
        games_classification,
        drop(["average_rating", 'NumOwned', 'NumWish', 'NumWant'], axis=1),
        games_classification['average_rating'],
        train_size=0.8,
        stratify = games_classification["average_rating"],
        random_state = 42)
```

```
[37]: reg_df_train, reg_df_test, reg_labels_train, reg_labels_test=train_test_split(
        games_regression.drop(["average_rating", 'NumOwned', 'NumWish', 'NumWant'], \
        axis=1),
        games_regression['average_rating'],
        train_size=0.8,
        random_state = 42)
```

```
[38]: ratings['total_ratings'].plot(kind='box')
```

[38]: <Axes: >



[39]: class_df_train

```
[39]:
```

	BGGId	Name	YearPublished	\
8120	32412	Palastgeflüster	2007	
10392	89222	Bloodsuckers	2011	
8740	38826	San Juan Hill: The Santiago Campaign 1898	2009	
21857	340790	1923 Cotton Club	2021	
6568	19475	Spingo	1980	
...	...	...	...	
18997	255615	GoodCritters	2018	
16804	213871	Yippie Yippie Yeti	2016	
9622	63710	Silk Road Maker	2009	
21489	317432	Exit: The Game - The Cursed Labyrinth	2021	
1344	1823	1829	1974	

	MfgPlaytime	MinPlayers	MaxPlayers	MfgAgeRec	NumUserRatings
8120	45	3	5	10	2268
10392	120	2	4	13	129
8740	60	2	2	12	40
21857	60	2	4	12	126

6568	20	2	8	8	50
...	...	...	...	...	...
18997	45	4	8	12	1028
16804	45	2	4	6	31
9622	10	1	4	7	37
21489	90	1	4	10	161
1344	360	3	9	12	233

[17540 rows x 8 columns]

KNN

```
[40]: #
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import GridSearchCV
classifier = KNeighborsClassifier()
parameters={'n_neighbors':range(1,25)}
gridsearch=GridSearchCV(classifier, parameters, cv=10, verbose=1)
gridsearch.fit(class_df_train.drop(["BGGId", "Name",
↪ "NumUserRatings"],axis=1),class_labels_train)
print("Best parametre: ", gridsearch.best_estimator_)
```

Fitting 10 folds for each of 24 candidates, totalling 240 fits

```
C:\Users\op101\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\sklearn\model_selection\_split.py:805: UserWarning: The least populated class in y has only 6 members, which is less than n_splits=10.
warnings.warn(
```

Best parametre: KNeighborsClassifier(n_neighbors=24)

```
[41]: classifier = KNeighborsClassifier()
parameters={'n_neighbors':range(25,50)}
gridsearch=GridSearchCV(classifier, parameters, cv=10, verbose=1)
gridsearch.fit(class_df_train.drop(["BGGId", "Name",
↪ "NumUserRatings"],axis=1),class_labels_train)
print("Best parametre: ", gridsearch.best_estimator_)
```

Fitting 10 folds for each of 25 candidates, totalling 250 fits

```
C:\Users\op101\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\sklearn\model_selection\_split.py:805: UserWarning: The least populated class in y has only 6 members, which is less than n_splits=10.
warnings.warn(
```

Best parametre: KNeighborsClassifier(n_neighbors=49)

```
[42]: from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
classifier = KNeighborsClassifier()
parameters={'n_neighbors':range(1,50)}
gridsearch=GridSearchCV(classifier, parameters, cv=10, verbose=1)
gridsearch.fit(scaler.fit_transform(class_df_train.drop(["BGGId", "Name",
↪ "NumUserRatings"],axis=1)),class_labels_train)
print("Best parametre: ", gridsearch.best_estimator_)
```

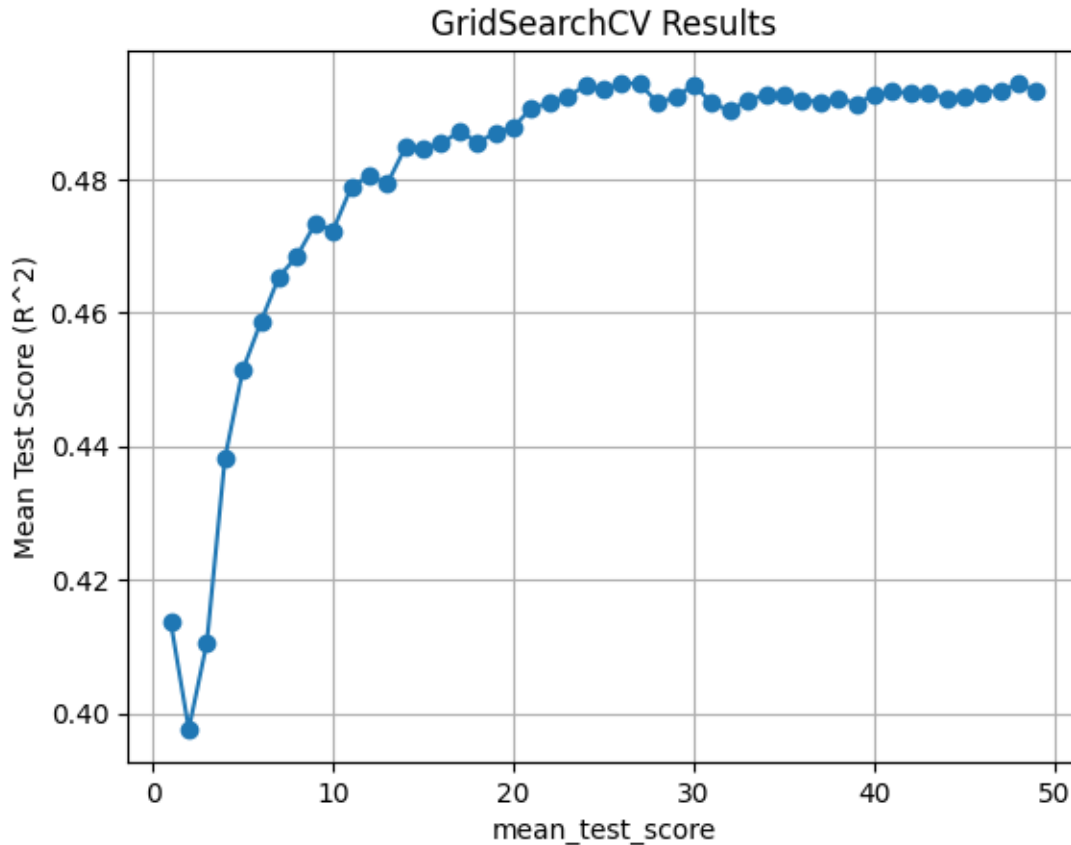
Fitting 10 folds for each of 49 candidates, totalling 490 fits

```
C:\Users\op101\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\sklearn\model_selection\_split.py:805: UserWarning: The least populated class in y has only 6 members, which is less than n_splits=10.
  warnings.warn(
```

Best parametre: KNeighborsClassifier(n_neighbors=26)

```
[43]: param_values=gridsearch.cv_results_['param_n_neighbors']
mean_test_scores=gridsearch.cv_results_['mean_test_score']

plt.plot(param_values, mean_test_scores, marker='o')
plt.xlabel('mean_test_score')
plt.ylabel('Mean Test Score (R^2)')
plt.title('GridSearchCV Results')
plt.grid(True)
plt.show()
```



```
[44]: knn=gridsearch.best_estimator_
X_train=scaler.fit_transform(class_df_train.drop(["BGGId", "Name",
↪ "NumUserRatings"],axis=1))
y_train=class_labels_train
X_test=scaler.transform(class_df_test.drop(["BGGId", "Name",
↪ "NumUserRatings"],axis=1))
y_test=class_labels_test
knn.fit(X_train, y_train)
print("Accuracy of KNN on training data:", round(knn.score(X_train, y_train),
↪ 5))
print("Accuracy of KNN on test data:", round(knn.score(X_test, y_test), 5))
```

Accuracy of KNN on training data: 0.53307

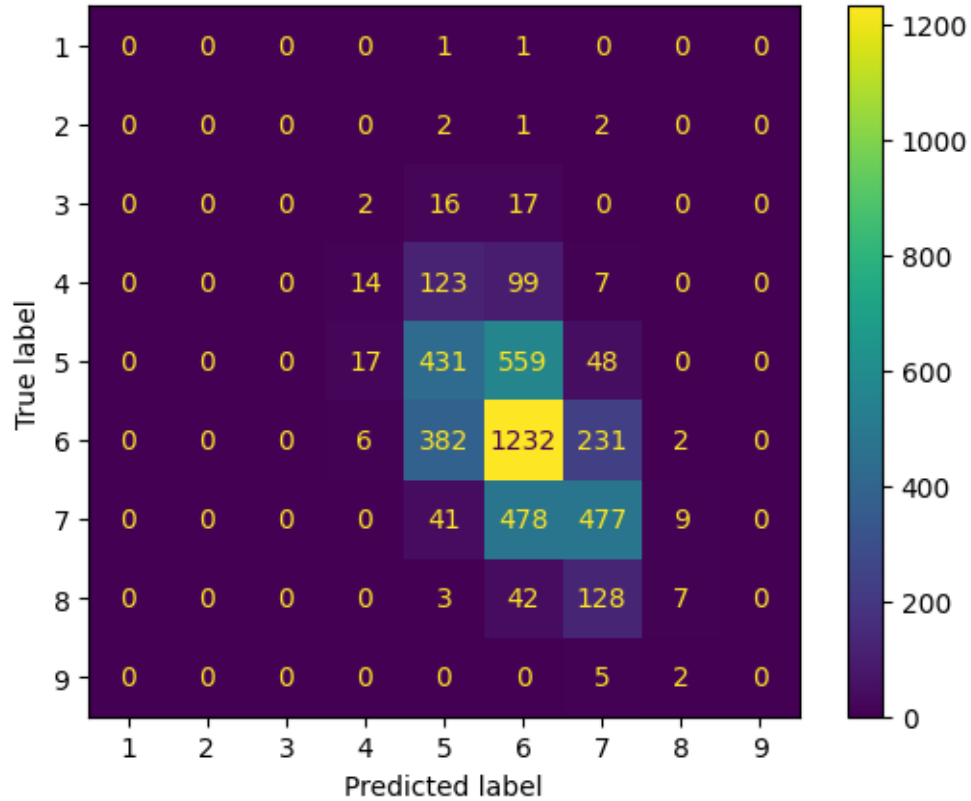
Accuracy of KNN on test data: 0.49282

KNN

```
[45]: from sklearn.metrics import classification_report, confusion_matrix,
↪ ConfusionMatrixDisplay
cm=confusion_matrix(y_test, knn.predict(X_test), labels=knn.classes_)
```

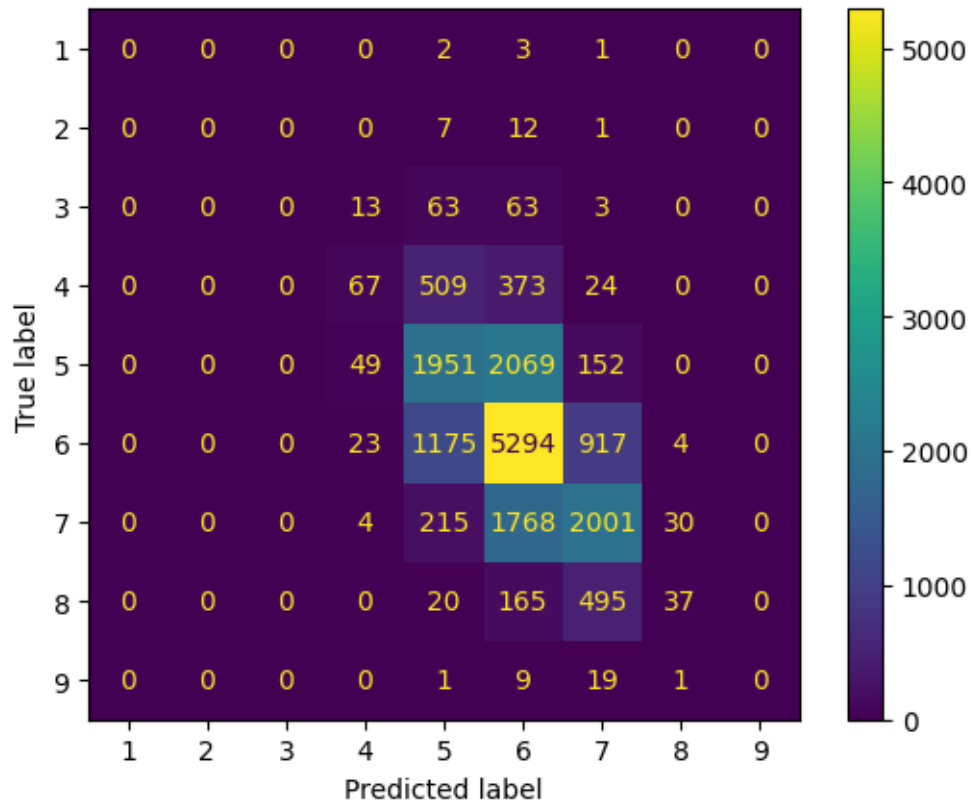
```
disp=ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=knn.classes_)
disp.plot()
```

[45]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x20f3c5bbd10>



```
[46]: cm=confusion_matrix(y_train, knn.predict(X_train), labels=knn.classes_)
disp=ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=knn.classes_)
disp.plot()
```

[46]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x20f27aaa650>



SVC

```
[47]: from sklearn.svm import SVC
svc=SVC(random_state=0)
X_train=class_df_train.drop(["BGGId", "Name", "NumUserRatings"],axis=1)
X_test=class_df_test.drop(["BGGId", "Name", "NumUserRatings"],axis=1)
svc.fit(X_train,y_train)
print("Accuracy of SVC on train:", round(svc.score(X_train,y_train),5))
print("Accuracy of SVC on test:", round(svc.score(X_test, y_test),5))
```

Accuracy of SVC on train: 0.42868

Accuracy of SVC on test: 0.43124

```
[48]: X_train=scaler.fit_transform(class_df_train.drop(["BGGId", "Name",
↪ "NumUserRatings"],axis=1))
X_test=scaler.transform(class_df_test.drop(["BGGId", "Name",
↪ "NumUserRatings"],axis=1))

svc=SVC(random_state=0)
svc.fit(X_train,y_train)
print("Accuracy of SVC on train:", round(svc.score(X_train,y_train),5))
print("Accuracy of SVC on test:", round(svc.score(X_test, y_test),5))
```

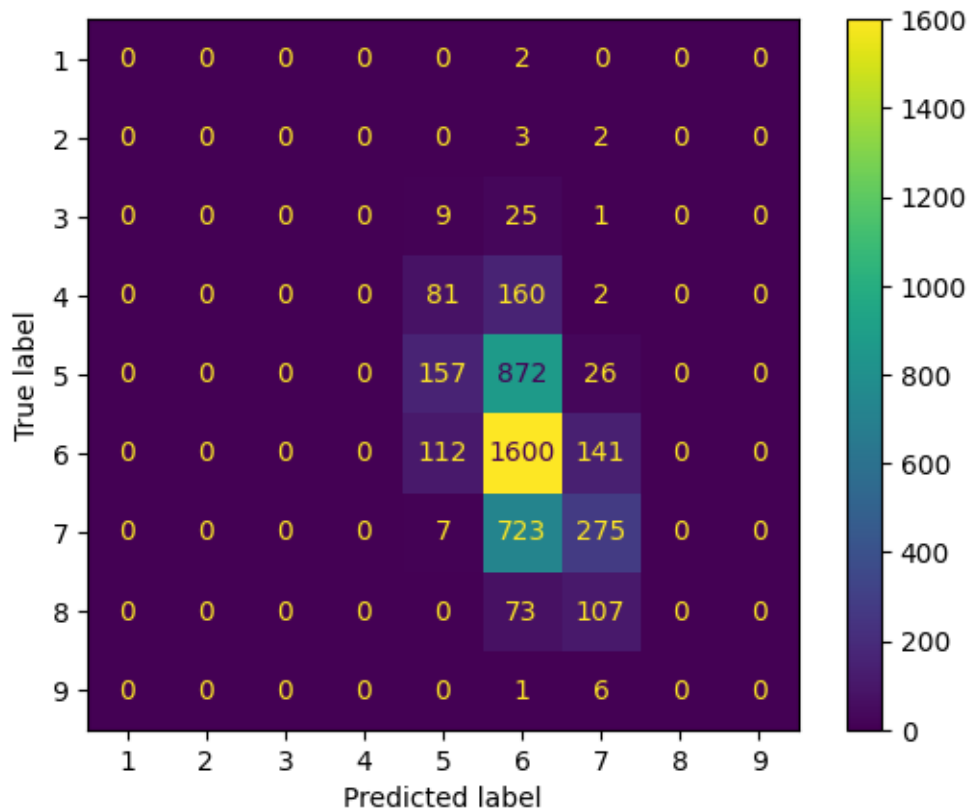
Accuracy of SVC on train: 0.47212
Accuracy of SVC on test: 0.4634

```
[49]: svc_save=svc
```

SVC

```
[50]: cm=confusion_matrix(y_test, svc.predict(X_test), labels=svc.classes_)
disp=ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=svc.classes_)
disp.plot()
```

```
[50]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x20f25ca5050>
```



```
[51]: svc=SVC(kernel="poly",random_state=0)
svc.fit(X_train,y_train)
print("Accuracy of SVC on train:", round(svc.score(X_train,y_train),5))
print("Accuracy of SVC on test:", round(svc.score(X_test, y_test),5))
```

Accuracy of SVC on train: 0.44265
Accuracy of SVC on test: 0.43512

C

```
[52]: import time
for c in [1, 10, 100]:
    start = time.time()
    model = SVC(C=c, kernel='linear') # or 'rbf'
    model.fit(X_train, y_train)
    print(f"C={c} took {time.time() - start:.2f} sec")

    print("Accuracy of SVC on train:", round(model.score(X_train, y_train), 5))
    print("Accuracy of SVC on test:", round(model.score(X_test, y_test), 5))
```

C=1 took 25.24 sec

Accuracy of SVC on train: 0.42417

Accuracy of SVC on test: 0.42349

```
-----
KeyboardInterrupt                                Traceback (most recent call last)
Cell In[52], line 5
      3 start = time.time()
      4 model = SVC(C=c, kernel='linear') # or 'rbf'
----> 5 model.fit(X_train, y_train)
      6 print(f"C={c} took {time.time() - start:.2f} sec")
      8 print("Accuracy of SVC on train:", round(model.score(X_train, y_train), 5))

File ~\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.
~\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\sklearn\base.py:1389, in _fit_context.<locals>.decorator.<locals>.wrapper(estimator, *args, **kwargs)
    1382 estimator._validate_params()
    1384 with config_context(
    1385     skip_parameter_validation=(
    1386         prefer_skip_nested_validation or global_skip_validation
    1387     ):
    1388 ):
-> 1389     return fit_method(estimator, *args, **kwargs)

File ~\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.
~\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\sklearn\svm\_base.py:258, in BaseLibSVM.fit(self, X, y, sample_weight)
    255 print("[LibSVM]", end="")
    257 seed = rnd.randint(np.iinfo("i").max)
--> 258 fit(X, y, sample_weight, solver_type, kernel, random_seed=seed)
    259 # see comment on the other call to np.iinfo in this file
    261 self.shape_fit_ = X.shape if hasattr(X, "shape") else (n_samples,)

File ~\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.
~\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\sklearn\svm\_base.py:336, in BaseLibSVM._dense_fit(self, X, y, sample_weight, solver_type, kernel, random_seed)
    322 libsvm.set_verbosity_wrap(self.verbose)
```

```

324 # we don't pass **self.get_params() to allow subclasses to
325 # add other parameters to __init__
326 (
327     self.support_,
328     self.support_vectors_,
329     self._n_support,
330     self.dual_coef_,
331     self.intercept_,
332     self._probA,
333     self._probB,
334     self.fit_status_,
335     self._num_iter,
--> 336 ) = libsvm.fit(
337     X,
338     y,
339     svm_type=solver_type,
340     sample_weight=sample_weight,
341     class_weight=getattr(self, "class_weight_", np.empty(0)),
342     kernel=kernel,
343     C=self.C,
344     nu=self.nu,
345     probability=self.probability,
346     degree=self.degree,
347     shrinking=self.shrinking,
348     tol=self.tol,
349     cache_size=self.cache_size,
350     coef0=self.coef0,
351     gamma=self._gamma,
352     epsilon=self.epsilon,
353     max_iter=self.max_iter,
354     random_seed=random_seed,
355 )
357 self._warn_from_fit_status()

```

KeyboardInterrupt:

```

[53]: from sklearn.ensemble import RandomForestClassifier
forest=RandomForestClassifier(max_depth=5)

```

```

[54]: X_train=scaler.fit_transform(class_df_train.drop(["BGGId", "Name",
↪ "NumUserRatings"],axis=1))
y_train=class_labels_train
X_test=scaler.transform(class_df_test.drop(["BGGId", "Name",
↪ "NumUserRatings"],axis=1))
y_test=class_labels_test

```

```

forest.fit(X_train, y_train)
print("Accuracy of Random Forest on training data:", round(forest.
    ↪score(X_train, y_train), 5))
print("Accuracy of Random Forest on test data:", round(forest.score(X_test,
    ↪y_test), 5))

```

Accuracy of Random Forest on training data: 0.50718

Accuracy of Random Forest on test data: 0.50262

RandomForest

```

[55]: param_grid = {
    'n_estimators': [100, 300, 500],
    'max_depth': [None, 10, 20, 30],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 5],
    'max_features': ['sqrt', 'log2'],
    'class_weight': [None, 'balanced']
}

clf = RandomForestClassifier(random_state=42)
grid = GridSearchCV(clf, param_grid, scoring='accuracy', cv=5, n_jobs=-1)
grid.fit(X_train, y_train)

print("Best params:", grid.best_params_)

```

Best params: {'class_weight': None, 'max_depth': 10, 'max_features': 'sqrt',
'min_samples_leaf': 5, 'min_samples_split': 2, 'n_estimators': 300}

RandomForest

```

[56]: forest_best=RandomForestClassifier(class_weight=None, max_depth=10,
    max_features="sqrt", min_samples_leaf=5,
    min_samples_split=2, n_estimators=300)

forest_best.fit(X_train, y_train)
forest_save=forest_best
print("Accuracy of best Random Forest on training data:", round(forest_best.
    ↪score(X_train, y_train), 5))
print("Accuracy of best Random Forest on test data:", round(forest_best.
    ↪score(X_test, y_test), 5))

```

Accuracy of best Random Forest on training data: 0.56465

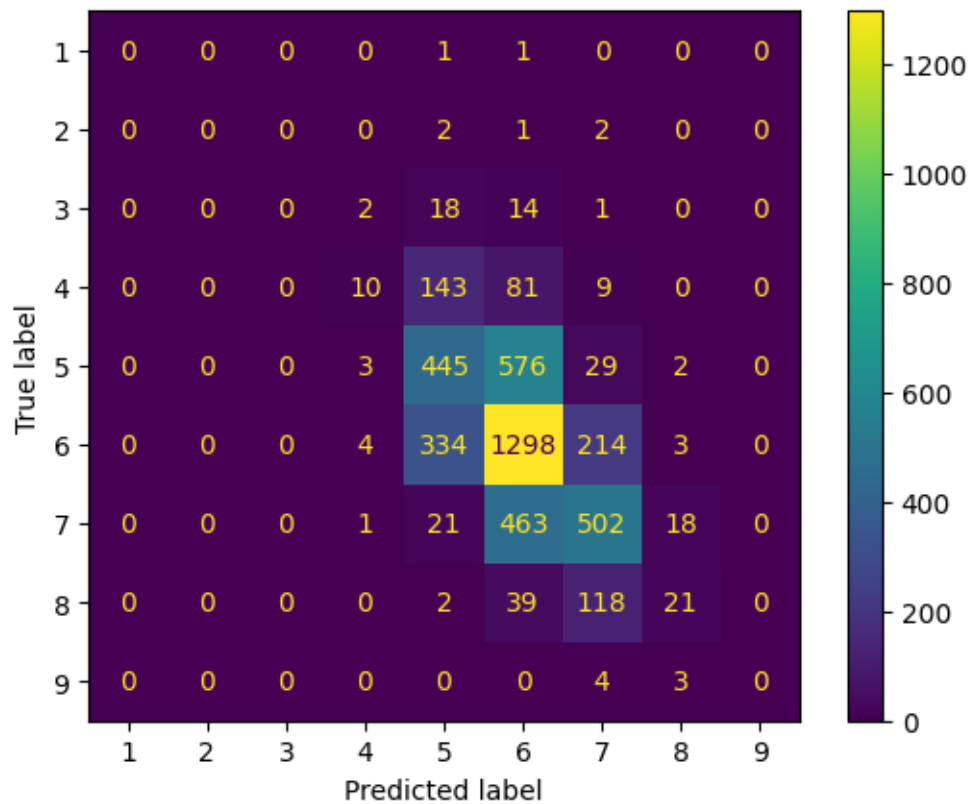
Accuracy of best Random Forest on test data: 0.51904

```

[57]: cm=confusion_matrix(y_test, forest_best.predict(X_test), labels=forest_best.
    ↪classes_)
disp=ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=forest_best.
    ↪classes_)
disp.plot()

```

```
[57]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x20f49ae24d0>
```



```
[58]: datasets['subcategory'].head()
```

	BGGId	Exploration	Miniatures	Territory Building	Card Game	Educational	\
0	1	0	0	0	0	0	
1	2	0	0	0	1	0	
2	3	0	0	0	0	0	
3	4	0	0	0	0	0	
4	5	0	0	1	0	0	
	Puzzle	Collectible Components	Word Game	Print & Play	Electronic		
0	0	0	0	0	0		
1	0	0	0	0	0		
2	0	0	0	0	0		
3	0	0	0	0	0		
4	0	0	0	0	0		

()

```
[59]: X_train=pd.merge(
        class_df_train,
        datasets['subcategory'],
        on='BGGId',
        how='inner'
    )
    X_train=scaler.fit_transform(X_train.drop(["BGGId", "Name", "NumUserRatings"],axis=1))
    y_train=class_labels_train
    X_test=pd.merge(
        class_df_test,
        datasets['subcategory'],
        on='BGGId',
        how='inner'
    )
    X_test=scaler.transform(X_test.drop(["BGGId", "Name", "NumUserRatings"],axis=1))
    y_test=class_labels_test
```

```
[60]: forest_best=RandomForestClassifier(class_weight=None, max_depth=20,
        max_features="sqrt", min_samples_leaf=5,
        min_samples_split=2, n_estimators=300)

    forest_best.fit(X_train, y_train)
    print("Accuracy of best Random Forest on training data:", round(forest_best.
        ↪score(X_train, y_train), 5))
    print("Accuracy of best Random Forest on test data:", round(forest_best.
        ↪score(X_test, y_test), 5))
```

Accuracy of best Random Forest on training data: 0.58706

Accuracy of best Random Forest on test data: 0.51995

```
[61]: X_train=class_df_train
    X_test=class_df_test
    for data_name in datasets.keys():
        X_train=pd.merge(
            X_train,
            datasets[data_name],
            on='BGGId',
            how='inner'
        )

        X_test=pd.merge(
            X_test,
            datasets[data_name],
            on='BGGId',
            how='inner'
        )
```

```

feat_names = X_train.drop(["BGGId", "Name", "NumUserRatings"],axis=1).columns

X_train=scaler.fit_transform(X_train.drop(["BGGId", "Name", "NumUserRatings"],axis=1))
y_train=class_labels_train
X_test=scaler.transform(X_test.drop(["BGGId", "Name", "NumUserRatings"],axis=1))
y_test=class_labels_test

```

```

[62]: forest_best=RandomForestClassifier(class_weight=None, max_depth=20,
                                         max_features="sqrt", min_samples_leaf=5,
                                         min_samples_split=2, n_estimators=300)

forest_best.fit(X_train, y_train)
print("Accuracy of best Random Forest on training data:", round(forest_best.
    ↳score(X_train, y_train), 5))
print("Accuracy of best Random Forest on test data:", round(forest_best.
    ↳score(X_test, y_test), 5))

```

Accuracy of best Random Forest on training data: 0.4866

Accuracy of best Random Forest on test data: 0.47662

```

[63]: X_test.size/y_test.size

```

```

[63]: 5527.0

```

```

[64]: X_train_complete=X_train
      X_test_complete=X_test

```

```

[65]: from sklearn.ensemble import AdaBoostClassifier
      from sklearn.ensemble import GradientBoostingClassifier
      from sklearn.tree import DecisionTreeClassifier

```

```

[66]: base_tree = DecisionTreeClassifier(max_depth=3)

ada = AdaBoostClassifier(
    estimator=base_tree,
    n_estimators=100,
    learning_rate=0.5,
    random_state=42
)

ada.fit(X_train, y_train)

base_tree = DecisionTreeClassifier(max_depth=3)
base_tree.fit(X_train, y_train)
print("DecisionTree train accuracy:", base_tree.score(X_train, y_train))

```

```

print("DecisionTree test accuracy:", base_tree.score(X_test, y_test))

print("AdaBoost train accuracy:", ada.score(X_train, y_train))
print("AdaBoost test accuracy:", ada.score(X_test, y_test))

```

DecisionTree train accuracy: 0.4771949828962372
 DecisionTree test accuracy: 0.47229190421892814
 AdaBoost train accuracy: 0.47702394526795894
 AdaBoost test accuracy: 0.47069555302166477

[67]: `base_tree = DecisionTreeClassifier(max_depth=3)`

```

grad = GradientBoostingClassifier(
    n_estimators=300,          # number of boosting rounds
    learning_rate=0.1,         # shrinkage factor
    max_depth=5,               # depth of each tree
    min_samples_split=2,       # minimum samples to split node
    min_samples_leaf=5,        # regularization
    subsample=0.8,             # stochastic gradient boosting
    max_features='sqrt',       # feature subsampling
    random_state=42
)

grad.fit(X_train, y_train)

base_tree = DecisionTreeClassifier(max_depth=3)
base_tree.fit(X_train, y_train)
print("DecisionTree train accuracy:", base_tree.score(X_train, y_train))
print("DecisionTree test accuracy:", base_tree.score(X_test, y_test))

print("GradBoost train accuracy:", grad.score(X_train, y_train))
print("GradBoost test accuracy:", grad.score(X_test, y_test))

```

DecisionTree train accuracy: 0.4771949828962372
 DecisionTree test accuracy: 0.47229190421892814
 GradBoost train accuracy: 0.2441847206385405
 GradBoost test accuracy: 0.23990877993158494

[68]: `y_train_xg=y_train-1`
`y_test_xg=y_test-1`

```

from xgboost import XGBClassifier

xgb = XGBClassifier(
    n_estimators=400,

```

```

max_depth=5,
learning_rate=0.1,
subsample=0.8,
colsample_bytree=0.8,
use_label_encoder=False,
eval_metric='mlogloss',      # for multiclass
random_state=42
)

xgb.fit(X_train, y_train_xg)

print("XGBoost train accuracy:", xgb.score(X_train, y_train_xg))
print("XGBoost test accuracy:", xgb.score(X_test, y_test_xg))

```

C:\Users\op101\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\xgboost\training.py:183: UserWarning: [22:24:36] WARNING: C:\actions-runner_work\xgboost\xgboost\src\learner.cc:738: Parameters: { "use_label_encoder" } are not used.

```

bst.update(dtrain, iteration=i, fobj=obj)

XGBoost train accuracy: 0.7151083238312429
XGBoost test accuracy: 0.5808437856328392

```

[69]: `from sklearn.metrics import classification_report`

```

y_pred = xgb.predict(X_test)
print(classification_report(y_test_xg, y_pred))

```

	precision	recall	f1-score	support
0	0.00	0.00	0.00	2
1	0.00	0.00	0.00	5
2	0.00	0.00	0.00	35
3	0.49	0.16	0.24	243
4	0.53	0.49	0.51	1055
5	0.58	0.74	0.65	1853
6	0.64	0.57	0.60	1005
7	0.60	0.32	0.42	180
8	0.00	0.00	0.00	7
accuracy			0.58	4385
macro avg	0.32	0.25	0.27	4385
weighted avg	0.57	0.58	0.57	4385

C:\Users\op101\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-

```

packages\sklearn\metrics\_classification.py:1565: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\Users\op101\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n
2kfra8p0\LocalCache\local-packages\Python311\site-
packages\sklearn\metrics\_classification.py:1565: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\Users\op101\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n
2kfra8p0\LocalCache\local-packages\Python311\site-
packages\sklearn\metrics\_classification.py:1565: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

```

```

[70]: def xgb_boost_features(features):
    X_train=class_df_train
    X_test=class_df_test

    for data_name in features:
        X_train=pd.merge(
            X_train,
            datasets[data_name],
            on='BGGId',
            how='inner'
        )

        X_test=pd.merge(
            X_test,
            datasets[data_name],
            on='BGGId',
            how='inner'
        )

    feat_names = X_train.drop(["BGGId", "Name", "NumUserRatings"],axis=1).
↳columns

    X_train=scaler.fit_transform(X_train.drop(["BGGId", "Name",
↳"NumUserRatings"],axis=1))
    y_train=class_labels_train
    X_test=scaler.transform(X_test.drop(["BGGId", "Name",
↳"NumUserRatings"],axis=1))
    y_test=class_labels_test

    y_train_xg=y_train-1

```

```

y_test_xg=y_test-1

xgb = XGBClassifier(
    n_estimators=100,
    max_depth=5,
    learning_rate=0.1,
    subsample=0.8,
    colsample_bytree=0.8,
    use_label_encoder=False,
    eval_metric='mlogloss',      # for multiclass
    random_state=42
)

xgb.fit(X_train, y_train_xg)

print (features)
print("XGBoost train accuracy:", xgb.score(X_train, y_train_xg))
print("XGBoost test accuracy:", xgb.score(X_test, y_test_xg))
print('\n')
return xgb

```

```
[71]: xgb_total=xgb
```

```

[72]: import itertools
array=['mechanic','subcategory','themes', 'publisher']
for L in range(len(array) + 1):
    for subset in itertools.combinations(array, L):
        xgb_boost_features(subset)

```

C:\Users\op101\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\xgboost\training.py:183: UserWarning: [22:32:43] WARNING: C:\actions-runner_work\xgboost\xgboost\src\learner.cc:738: Parameters: { "use_label_encoder" } are not used.

```

    bst.update(dtrain, iteration=i, fobj=obj)
()
XGBoost train accuracy: 0.5529076396807298
XGBoost test accuracy: 0.5167616875712657

```

C:\Users\op101\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\xgboost\training.py:183: UserWarning: [22:32:45] WARNING: C:\actions-runner_work\xgboost\xgboost\src\learner.cc:738: Parameters: { "use_label_encoder" } are not used.

```
bst.update(dtrain, iteration=i, fobj=obj)
('mechanic',)
XGBoost train accuracy: 0.5866020524515393
XGBoost test accuracy: 0.5438996579247435
```

```
C:\Users\op101\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\xgboost\training.py:183: UserWarning: [22:32:52] WARNING: C:\actions-runner\_work\xgboost\xgboost\src\learner.cc:738:
Parameters: { "use_label_encoder" } are not used.
```

```
bst.update(dtrain, iteration=i, fobj=obj)
('subcategory',)
XGBoost train accuracy: 0.5618015963511973
XGBoost test accuracy: 0.5217787913340936
```

```
C:\Users\op101\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\xgboost\training.py:183: UserWarning: [22:32:55] WARNING: C:\actions-runner\_work\xgboost\xgboost\src\learner.cc:738:
Parameters: { "use_label_encoder" } are not used.
```

```
bst.update(dtrain, iteration=i, fobj=obj)
('themes',)
XGBoost train accuracy: 0.572405929304447
XGBoost test accuracy: 0.5229190421892816
```

```
C:\Users\op101\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\xgboost\training.py:183: UserWarning: [22:33:14] WARNING: C:\actions-runner\_work\xgboost\xgboost\src\learner.cc:738:
Parameters: { "use_label_encoder" } are not used.
```

```
bst.update(dtrain, iteration=i, fobj=obj)
('publisher',)
XGBoost train accuracy: 0.5960091220068415
XGBoost test accuracy: 0.5486887115165336
```

```
C:\Users\op101\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-
```

```
packages\xgboost\training.py:183: UserWarning: [22:34:03] WARNING: C:\actions-  
runner\_work\xgboost\xgboost\src\learner.cc:738:  
Parameters: { "use_label_encoder" } are not used.
```

```
    bst.update(dtrain, iteration=i, fobj=obj)  
  
('mechanic', 'subcategory')  
XGBoost train accuracy: 0.5945838084378563  
XGBoost test accuracy: 0.5395667046750285
```

```
C:\Users\op101\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n  
2kfra8p0\LocalCache\local-packages\Python311\site-  
packages\xgboost\training.py:183: UserWarning: [22:34:10] WARNING: C:\actions-  
runner\_work\xgboost\xgboost\src\learner.cc:738:  
Parameters: { "use_label_encoder" } are not used.
```

```
    bst.update(dtrain, iteration=i, fobj=obj)  
  
('mechanic', 'themes')  
XGBoost train accuracy: 0.5957240592930445  
XGBoost test accuracy: 0.5409350057012543
```

```
C:\Users\op101\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n  
2kfra8p0\LocalCache\local-packages\Python311\site-  
packages\xgboost\training.py:183: UserWarning: [22:34:27] WARNING: C:\actions-  
runner\_work\xgboost\xgboost\src\learner.cc:738:  
Parameters: { "use_label_encoder" } are not used.
```

```
    bst.update(dtrain, iteration=i, fobj=obj)  
  
('mechanic', 'publisher')  
XGBoost train accuracy: 0.6130558722919042  
XGBoost test accuracy: 0.560091220068415
```

```
C:\Users\op101\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n  
2kfra8p0\LocalCache\local-packages\Python311\site-  
packages\xgboost\training.py:183: UserWarning: [22:35:17] WARNING: C:\actions-  
runner\_work\xgboost\xgboost\src\learner.cc:738:  
Parameters: { "use_label_encoder" } are not used.
```

```
    bst.update(dtrain, iteration=i, fobj=obj)  
  
('subcategory', 'themes')  
XGBoost train accuracy: 0.5794754846066135  
XGBoost test accuracy: 0.5233751425313569
```

```
C:\Users\op101\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\xgboost\training.py:183: UserWarning: [22:35:30] WARNING: C:\actions-runner\_work\xgboost\xgboost\src\learner.cc:738:
Parameters: { "use_label_encoder" } are not used.
```

```
bst.update(dtrain, iteration=i, fobj=obj)
('subcategory', 'publisher')
XGBoost train accuracy: 0.5986887115165337
XGBoost test accuracy: 0.5498289623717217
```

```
C:\Users\op101\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\xgboost\training.py:183: UserWarning: [22:36:22] WARNING: C:\actions-runner\_work\xgboost\xgboost\src\learner.cc:738:
Parameters: { "use_label_encoder" } are not used.
```

```
bst.update(dtrain, iteration=i, fobj=obj)
('themes', 'publisher')
XGBoost train accuracy: 0.6047890535917902
XGBoost test accuracy: 0.5537058152793615
```

```
C:\Users\op101\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\xgboost\training.py:183: UserWarning: [22:37:17] WARNING: C:\actions-runner\_work\xgboost\xgboost\src\learner.cc:738:
Parameters: { "use_label_encoder" } are not used.
```

```
bst.update(dtrain, iteration=i, fobj=obj)
('mechanic', 'subcategory', 'themes')
XGBoost train accuracy: 0.6012542759407069
XGBoost test accuracy: 0.544127708095781
```

```
C:\Users\op101\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\xgboost\training.py:183: UserWarning: [22:37:35] WARNING: C:\actions-runner\_work\xgboost\xgboost\src\learner.cc:738:
Parameters: { "use_label_encoder" } are not used.
```

```

bst.update(dtrain, iteration=i, fobj=obj)

('mechanic', 'subcategory', 'publisher')
XGBoost train accuracy: 0.6149372862029646
XGBoost test accuracy: 0.5614595210946408

```

```

C:\Users\op101\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n
2kfra8p0\LocalCache\local-packages\Python311\site-
packages\xgboost\training.py:183: UserWarning: [22:38:30] WARNING: C:\actions-
runner\_work\xgboost\xgboost\src\learner.cc:738:
Parameters: { "use_label_encoder" } are not used.

```

```

bst.update(dtrain, iteration=i, fobj=obj)

('mechanic', 'themes', 'publisher')
XGBoost train accuracy: 0.617502850627138
XGBoost test accuracy: 0.5582668187001141

```

```

C:\Users\op101\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n
2kfra8p0\LocalCache\local-packages\Python311\site-
packages\xgboost\training.py:183: UserWarning: [22:39:36] WARNING: C:\actions-
runner\_work\xgboost\xgboost\src\learner.cc:738:
Parameters: { "use_label_encoder" } are not used.

```

```

bst.update(dtrain, iteration=i, fobj=obj)

('subcategory', 'themes', 'publisher')
XGBoost train accuracy: 0.608266818700114
XGBoost test accuracy: 0.552109464082098

```

```

C:\Users\op101\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n
2kfra8p0\LocalCache\local-packages\Python311\site-
packages\xgboost\training.py:183: UserWarning: [22:40:41] WARNING: C:\actions-
runner\_work\xgboost\xgboost\src\learner.cc:738:
Parameters: { "use_label_encoder" } are not used.

```

```

bst.update(dtrain, iteration=i, fobj=obj)

('mechanic', 'subcategory', 'themes', 'publisher')
XGBoost train accuracy: 0.623945267958951
XGBoost test accuracy: 0.5635119726339795

```

```

(           )           .           ( )           (
).

```

```
[73]: X_train=class_df_train
X_test=class_df_test
features=['mechanic', 'subcategory', 'themes', 'publisher']
for data_name in features:
    X_train=pd.merge(
        X_train,
        datasets[data_name],
        on='BGGId',
        how='inner'
    )

    X_test=pd.merge(
        X_test,
        datasets[data_name],
        on='BGGId',
        how='inner'
    )

X_test=scaler.transform(X_test.drop(["BGGId", "Name", "NumUserRatings"],axis=1))
y_test=class_labels_test
y_train_xg=y_train-1
y_test_xg=y_test-1
xgb=xgb_boost_features(features)
y_pred = xgb.predict(X_test)

print(classification_report(y_test_xg, y_pred))
```

C:\Users\op101\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\xgboost\training.py:183: UserWarning: [22:41:47] WARNING: C:\actions-runner_work\xgboost\xgboost\src\learner.cc:738: Parameters: { "use_label_encoder" } are not used.

```
bst.update(dtrain, iteration=i, fobj=obj)
['mechanic', 'subcategory', 'themes', 'publisher']
XGBoost train accuracy: 0.623945267958951
XGBoost test accuracy: 0.5635119726339795
```

	precision	recall	f1-score	support
0	0.00	0.00	0.00	2
1	0.00	0.00	0.00	5
2	0.00	0.00	0.00	35

3	0.51	0.09	0.16	243
4	0.51	0.45	0.48	1055
5	0.56	0.75	0.64	1853
6	0.61	0.55	0.58	1005
7	0.64	0.23	0.34	180
8	0.00	0.00	0.00	7
accuracy			0.56	4385
macro avg	0.32	0.23	0.24	4385
weighted avg	0.56	0.56	0.54	4385

```
C:\Users\op101\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\sklearn\metrics\_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
```

```
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

```
C:\Users\op101\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\sklearn\metrics\_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
```

```
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

```
C:\Users\op101\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\sklearn\metrics\_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
```

```
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

```
[74]: importances = xgb.feature_importances_
feat_names = X_train.drop(["BGGId", "Name", "NumUserRatings"],axis=1).columns

importance_df = pd.DataFrame({
    'feature': feat_names,
    'importance': importances
}).sort_values(by='importance', ascending=False)

print(importance_df.head(20))
```

	feature	importance
36	Roll / Spin and Move	0.006791
1891	Ogry Games	0.006191
789	uplay.it edizioni	0.005719
1371	Ghenos Games	0.005685
469	Filosofia Éditions	0.005495
0	YearPublished	0.005016

163	Miniatures	0.004795
389	Hans im Glück	0.004470
92	Solo / Solitaire Game	0.004425
50	Simulation	0.004406
2249	Funko Games	0.004377
967	Victory Games (I)	0.004332
1026	Asterion Press	0.004236
1	MfgPlaytime	0.004175
133	Communication Limits	0.004081
401	Descartes Editeur	0.003990
1426	Banana Games	0.003870
1167	Kadon Enterprises	0.003784
2035	Palm Court	0.003670
781	nestorgames	0.003658

```
[75]: importances = forest_best.feature_importances_
      feat_names=forest_best.feature_names_in_

      importance_df = pd.DataFrame({
          'feature': feat_names,
          'importance': importances
      }).sort_values(by='importance', ascending=False)

      print(importance_df.head(20))
```

```
-----
AttributeError                                Traceback (most recent call last)
Cell In[75], line 2
      1 importances = forest_best.feature_importances_
----> 2 feat_names=forest_best.feature_names_in_
      4 importance_df = pd.DataFrame({
      5     'feature': feat_names,
      6     'importance': importances
      7 }).sort_values(by='importance', ascending=False)
      9 print(importance_df.head(20))

AttributeError: 'RandomForestClassifier' object has no attribute_
↳ 'feature_names_in_'
```

```
[76]: artists_numbered=datasets['designer'].drop("BGGId",axis=1).sum(axis=0).
      ↳sort_values(ascending=False)
      artists_numbered[artists_numbered>0]
```

```
[76]: Low-Exp Designer      8355
      (Uncredited)         1452
      Reiner Knizia         338
```

Joseph Miranda	141
Wolfgang Kramer	136
...	
Łukasz Wrona	4
Tim Uren	4
Scott Kimball	4
Michael Stackpole	4
Max Wikström	4

Length: 1593, dtype: int64

```
[77]: artists_numbered=datasets['artist'].drop("BGGId",axis=1).sum(axis=0).
      ↪sort_values(ascending=False)
      artists_numbered
```

```
[77]: Low-Exp Artist          6989
      Rodger B. MacGowan      379
      (Uncredited)           363
      Franz Vohwinkel         318
      Redmond A. Simonsen     254
      ...
      Michael Hays            4
      Chris Drew              4
      Christopher Peacey      4
      Jonathan La Trobe-Lewis 4
      Johannes Vermeer        4
      Length: 1680, dtype: int64
```

```
[78]: artists_numbered[artists_numbered>100].size
```

```
[78]: 16
```

```
[79]: artists_numbered=datasets['publisher'].drop("BGGId",axis=1).sum(axis=0).
      ↪sort_values(ascending=False)
      artists_numbered[artists_numbered>0]
```

```
[79]: Low-Exp Publisher      4764
      Hasbro                 594
      (Self-Published)      592
      Asmodee                575
      Ravensburger           523
      ...
      Past Times             4
      2D6.EE (Side Quest Games) 4
      Dice Realm             4
      The Games Team Ltd.     4
      Games and Puzzles (magazine) 4
      Length: 1865, dtype: int64
```

```
[80]: artists_numbered=datasets['mechanic']
artists_numbered.head()
```

```
[80]:
```

	BGGId	Alliances	Area Majority / Influence	Auction/Bidding	Dice Rolling	\
0	1	1	1	1	1	
1	2	0	0	0	0	
2	3	0	1	0	0	
3	4	0	1	1	0	
4	5	0	0	0	0	

	Hand Management	Simultaneous Action Selection	Trick-taking	Hexagon Grid	\
0	1	1	0	0	
1	0	0	1	0	
2	1	0	0	1	
3	0	0	0	0	
4	1	0	0	0	

	Once-Per-Game Abilities	... Contracts	Passed Action Token	\
0	0	...	0	0
1	0	...	0	0
2	1	...	0	0
3	0	...	0	0
4	0	...	0	0

	King of the Hill	Action Retrieval	Force Commitment	Rondel	\
0	0	0	0	0	
1	0	0	0	0	
2	0	0	0	0	
3	0	0	0	0	
4	0	0	0	0	

	Automatic Resource Growth	Legacy Game	Dexterity	Physical
0	0	0	0	0
1	0	0	0	0
2	0	0	0	0
3	0	0	0	0
4	0	0	0	0

[5 rows x 158 columns]

```
[81]: artists_numbered=datasets['mechanic'].drop("BGGId",axis=1).sum(axis=0).
↳sort_values(ascending=False)
artists_numbered[artists_numbered>0]
```

```
[81]: Dice Rolling          6486
Hand Management          4496
Set Collection          2959
```

Variable Player Powers	2736
Hexagon Grid	2438
...	
Induction	10
Pattern Movement	10
Different Dice Movement	10
Order Counters	9
Passed Action Token	5

Length: 157, dtype: int64

```
[82]: artists_numbered=datasets['themes'].drop("BGId",axis=1).sum(axis=0).
      ↪sort_values(ascending=False)
      artists_numbered[artists_numbered<2].head()
```

```
[82]: Series([], dtype: int64)
```

```
[ ]:
```

```
[83]: artists_numbered=datasets['subcategory'].drop("BGId",axis=1).sum(axis=0).
      ↪sort_values(ascending=False)
      artists_numbered[artists_numbered>0]
```

```
[83]: Card Game          6480
      Miniatures        1089
      Exploration        897
      Puzzle            676
      Print & Play        604
      Territory Building  514
      Educational        502
      Word Game          484
      Collectible Components 359
      Electronic         205
      dtype: int64
```

```
[84]: artists_numbered.value_counts().sort_index(ascending=True)
```

```
[84]: 205      1
      359      1
      484      1
      502      1
      514      1
      604      1
      676      1
      897      1
      1089     1
      6480     1
      Name: count, dtype: int64
```

```
[85]: X_train_numeric=scaler.fit_transform(class_df_train.drop(["BGGId", "Name",
↪ "NumUserRatings"],axis=1))
X_test_numeric=scaler.fit_transform(class_df_test.drop(["BGGId", "Name",
↪ "NumUserRatings"],axis=1))

[86]: X_train=class_df_train
X_test=class_df_test

for data_name in features:
    X_train=pd.merge(
        X_train,
        datasets[data_name],
        on='BGGId',
        how='inner'
    )

    X_test=pd.merge(
        X_test,
        datasets[data_name],
        on='BGGId',
        how='inner'
    )

feat_names = X_train.drop(["BGGId", "Name", "NumUserRatings"],axis=1).columns

X_train=scaler.fit_transform(X_train.drop(["BGGId", "Name",
↪ "NumUserRatings"],axis=1))
y_train=class_labels_train
X_test=scaler.transform(X_test.drop(["BGGId", "Name", "NumUserRatings"],axis=1))
y_test=class_labels_test

y_train_xg=y_train-1
y_test_xg=y_test-1

[87]: print("Accuracy on training data for ")
print("KNN classifier ", int(knn.score(X_train_numeric, y_train)*100), '%',
↪ '\tSVC ', int(svc.score(X_train_numeric,y_train)*100), '%', '\tRandom forest
↪:', int(forest_save.score(X_train_numeric,y_train)*100), '%', '\tAdaBoost :
↪', int(ada.score(X_train_complete,y_train)*100), '%', '\tXGBoost(total) :',
↪int(xgb_total.score(X_train_complete,y_train-1)*100), '%',
↪'\tXGBoost(selected) :', int(xgb.score(X_train,y_train-1)*100), '%')
print("Accuracy on test data for ")
```

```
print("KNN classifier ", int(knn.score(X_test_numeric, y_test)*100), '%',
      '\tSVC ', int(svc.score(X_test_numeric,y_test)*100), '%', '\tRandom forest :',
      '\t', int(forest_save.score(X_test_numeric,y_test)*100), '%', '\tAdaBoost :',
      '\t', int(ada.score(X_test_complete,y_test)*100), '%', '\tXGBoost(total) :',
      '\t', int(xgb_total.score(X_test_complete,y_test-1)*100), '%',
      '\tXGBoost(selected) :', int(xgb.score(X_test,y_test-1)*100), '%')
```

Accuracy on training data for

KNN classifier 53 % SVC 44 % Random forest : 56 % AdaBoost : 47 %
 XGBoost(total) : 71 % XGBoost(selected) : 62 %

Accuracy on test data for

KNN classifier 47 % SVC 43 % Random forest : 49 % AdaBoost : 47 %
 XGBoost(total) : 58 % XGBoost(selected) : 56 %

```
[88]: fig, ax = plt.subplots(1,2, figsize=(12, 5))

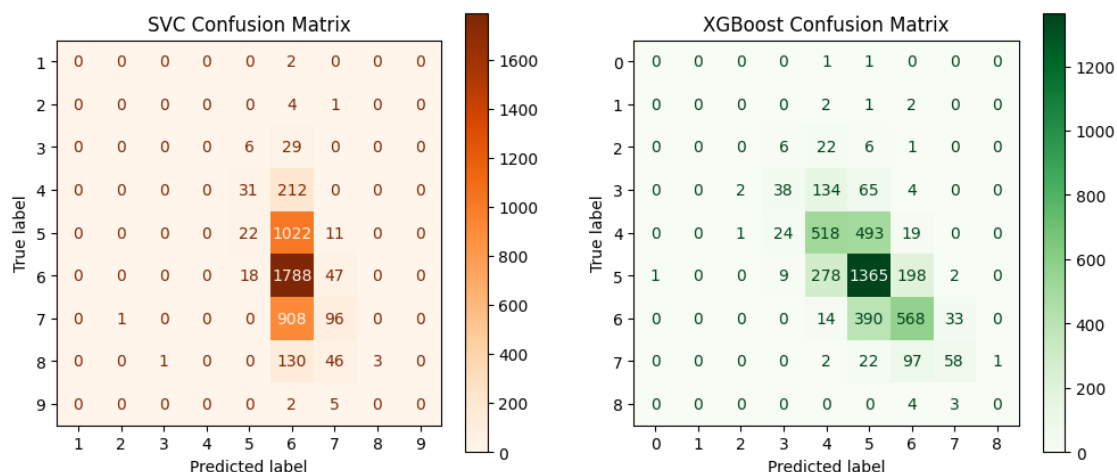
cm=confusion_matrix(y_test, svc.predict(X_test_numeric), labels=svc.classes_)
disp=ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=svc.classes_)
ax[0].set_title("SVC Confusion Matrix")

disp.plot(cmap="Oranges",ax=ax[0])

cm=confusion_matrix(y_test-1, xgb_total.predict(X_test_complete),
                    labels=xgb_total.classes_)
disp=ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=xgb_total.
                    classes_)

disp.plot(cmap="Greens",ax=ax[1])
ax[1].set_title("XGBoost Confusion Matrix")
```

[88]: Text(0.5, 1.0, 'XGBoost Confusion Matrix')



```
[89]: def subplot_matrix(df):
    """
    Creates a matrix of subplots:
    - Diagonal: bright KDE-enhanced histograms of each numeric parameter.
    - Lower triangle: scatter plots.
    - Upper triangle: correlation heatmap with coefficient values.

    Parameters:
    df: pandas DataFrame of features.
    """
    df_numeric = df.select_dtypes(include=[np.number])
    columns = df_numeric.columns
    n = len(columns)

    fig, axes = plt.subplots(n, n, figsize=(12, 12))
    plt.subplots_adjust(wspace=0.2, hspace=0.2)

    for i in range(n):
        for j in range(n):
            ax = axes[i, j]
            ax.clear()

            if i == j:
                # Diagonal: Bright-colored KDE-enhanced histogram
                sns.histplot(df_numeric[columns[i]], ax=ax, kde=True,
                    color="#ffcc00", edgecolor="black", linewidth=0.3)
                ax.set_xlim(df_numeric[columns[i]].min(),
                    df_numeric[columns[i]].max())

            elif i > j:
                # Lower triangle: Scatter plot
                ax.scatter(df_numeric[columns[j]], df_numeric[columns[i]],
                    alpha=0.5, s=10, color="teal")
                ax.set_xlim(df_numeric[columns[j]].min(),
                    df_numeric[columns[j]].max())
                ax.set_ylim(df_numeric[columns[i]].min(),
                    df_numeric[columns[i]].max())

            else:
                # Upper triangle: Correlation heatmap cell with value
                corr = np.corrcoef(df_numeric[columns[i]],
                    df_numeric[columns[j]])[0, 1]
```

```

        ax.imshow([[corr]], vmin=-1, vmax=1, cmap="coolwarm",
↪aspect="auto")
        ax.set_xticks([0])
        ax.set_yticks([0])
        ax.set_xticklabels([])
        ax.set_yticklabels([])
        ax.text(0.5, 0.5, f"{corr:.2f}", fontsize=11, ha='center',
↪va='center', color='black', transform=ax.transAxes)

    # Tick control
    if i < n - 1:
        ax.set_xticklabels([])
        ax.set_xticks([])
    else:
        ax.set_xlabel(columns[j], fontsize=9)
        ax.xaxis.set_tick_params(labelsize=7)

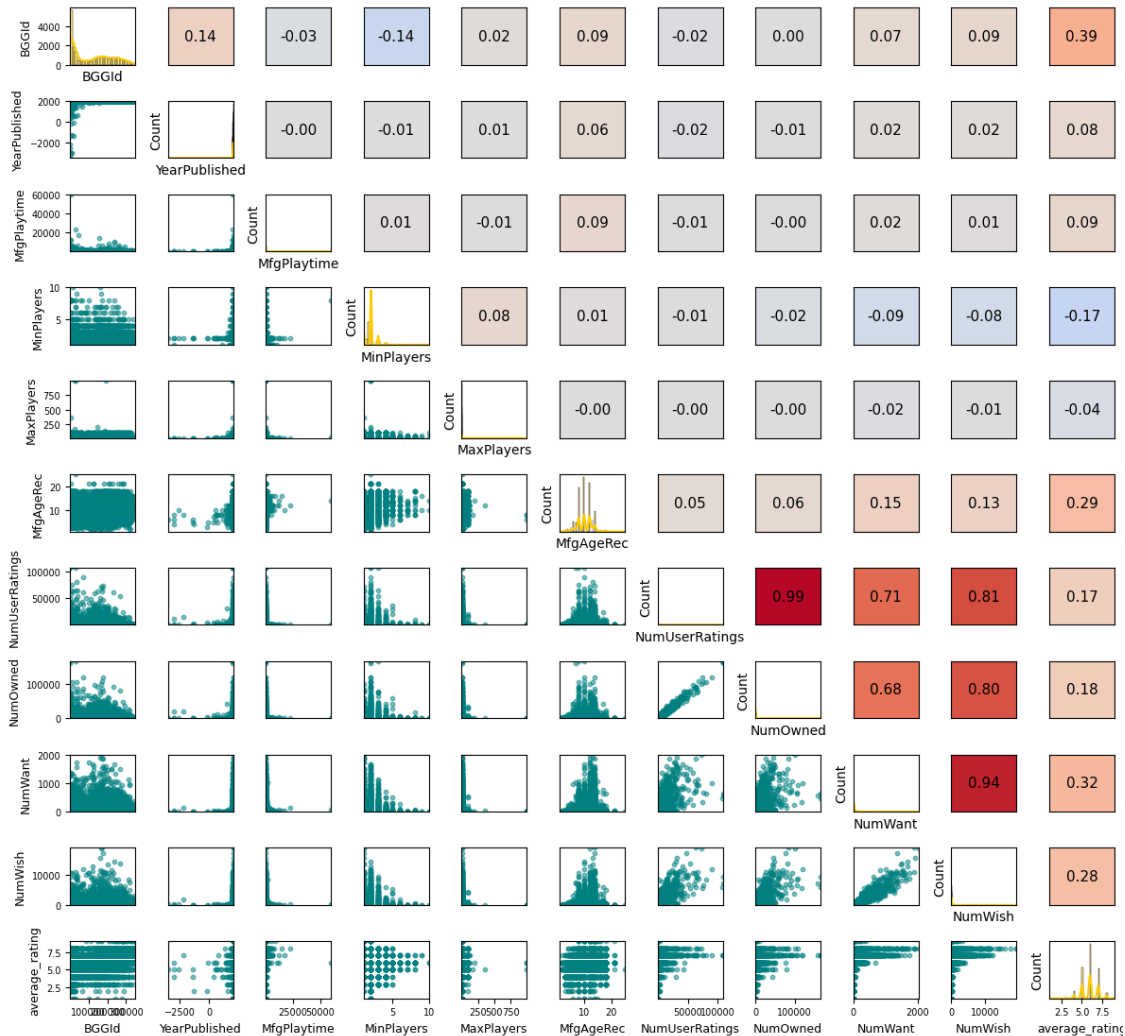
    if j > 0:
        ax.set_yticklabels([])
        ax.set_yticks([])
    else:
        ax.set_ylabel(columns[i], fontsize=9)
        ax.yaxis.set_tick_params(labelsize=7)

plt.suptitle("Pairwise Parameter Matrix", fontsize=16)
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()

```

```
[90]: subplot_matrix(games_classification)
```

Pairwise Parameter Matrix



```
[91]: #random sampling error
from sklearn.metrics import root_mean_squared_error, r2_score

y_random = np.random.choice(y_train, size=len(y_test), replace=True)

r2_random=r2_score(y_test,y_random)
print("R2 of random sampling baseline:", round(r2_random, 4))

rmse_random = root_mean_squared_error(y_test, y_random)
print("RMSE of random sampling baseline:", round(rmse_random, 4))
```

R2 of random sampling baseline: -0.9737

RMSE of random sampling baseline: 1.3816

```
[92]: X_train=reg_df_train.drop(['BGGId', 'NumUserRatings', 'Name'],axis=1)
X_test=reg_df_test.drop(['BGGId', 'NumUserRatings', 'Name'],axis=1)
y_train=reg_labels_train
y_test=reg_labels_test
```

```
[93]: def lin_reg_numeric_features(features):
    features=list(features)
    from sklearn.linear_model import LinearRegression
    linreg=LinearRegression()
    linreg.fit(reg_df_train[features], y_train)

    y_reg_predict=linreg.predict(reg_df_train[features])

    print(f"\n{features}")

    print(f"{name} Regression R2 train:", r2_score(y_train, y_reg_predict))
    print(f"{name} Regression RMSE train:", root_mean_squared_error(y_train,
↪y_reg_predict))

    y_reg_predict = linreg.predict(reg_df_test[features])

    print(f"{name} Regression R2 test:", r2_score(y_test, y_reg_predict))
    print(f"{name} Regression RMSE test:", root_mean_squared_error(y_test,
↪y_reg_predict))
```

```
[94]: import itertools
array=reg_df_train.drop(['BGGId', 'NumUserRatings', 'Name'],axis=1).columns
for L in range(len(array) + 1):
    for subset in itertools.combinations(array, L):
        if len(subset)>0:
            lin_reg_numeric_features(subset)
```

['YearPublished']

themes Regression R² train: 0.005450815899818551
themes Regression RMSE train: 0.929882025168584
themes Regression R² test: 0.0058521164445541896
themes Regression RMSE test: 0.9630630314770253

['MfgPlaytime']

themes Regression R² train: 0.023244710458830475
themes Regression RMSE train: 0.9215260276086694
themes Regression R² test: -0.16848773399719796
themes Regression RMSE test: 1.044098074740926

```
['MinPlayers']
themes Regression R2 train: 0.030117268780775364
themes Regression RMSE train: 0.9182783251645504
themes Regression R2 test: 0.03127726498620964
themes Regression RMSE test: 0.9506681896553202
```

```
['MaxPlayers']
themes Regression R2 train: 0.0012646782357010888
themes Regression RMSE train: 0.9318369444159097
themes Regression R2 test: 0.0027554254615620444
themes Regression RMSE test: 0.9645617973306203
```

```
['MfgAgeRec']
themes Regression R2 train: 0.0947641358746123
themes Regression RMSE train: 0.8871470156562515
themes Regression R2 test: 0.09366085378182576
themes Regression RMSE test: 0.9195483832878066
```

```
['YearPublished', 'MfgPlaytime']
themes Regression R2 train: 0.028694580750393994
themes Regression RMSE train: 0.9189515740036611
themes Regression R2 test: -0.16241957359220227
themes Regression RMSE test: 1.041383454163774
```

```
['YearPublished', 'MinPlayers']
themes Regression R2 train: 0.035433340534453794
themes Regression RMSE train: 0.9157582570446815
themes Regression R2 test: 0.036863471502949
themes Regression RMSE test: 0.9479231797441222
```

```
['YearPublished', 'MaxPlayers']
themes Regression R2 train: 0.006744109816348431
themes Regression RMSE train: 0.9292772275360248
themes Regression R2 test: 0.008661135930238872
themes Regression RMSE test: 0.9617014752448552
```

```
['YearPublished', 'MfgAgeRec']
themes Regression R2 train: 0.09800484663694742
themes Regression RMSE train: 0.8855576152037764
themes Regression R2 test: 0.09704772714267373
themes Regression RMSE test: 0.9178286578912978
```

```
['MfgPlaytime', 'MinPlayers']
themes Regression R2 train: 0.050701039232668554
themes Regression RMSE train: 0.9084817821163343
themes Regression R2 test: -0.1012400534356912
themes Regression RMSE test: 1.0136084337606341
```

```
['MfgPlaytime', 'MaxPlayers']
themes Regression R2 train: 0.024354662053002185
themes Regression RMSE train: 0.9210022833083296
themes Regression R2 test: -0.16469655937122085
themes Regression RMSE test: 1.0424029032945825
```

```
['MfgPlaytime', 'MfgAgeRec']
themes Regression R2 train: 0.10770748391336793
themes Regression RMSE train: 0.8807818268805568
themes Regression R2 test: -0.006169613699199106
themes Regression RMSE test: 0.9688684520425886
```

```
['MinPlayers', 'MaxPlayers']
themes Regression R2 train: 0.03060739522592093
themes Regression RMSE train: 0.9180462716638317
themes Regression R2 test: 0.03253658164577766
themes Regression RMSE test: 0.9500500656146675
```

```
['MinPlayers', 'MfgAgeRec']
themes Regression R2 train: 0.12603962015050185
themes Regression RMSE train: 0.8716870467734267
themes Regression R2 test: 0.1266046191943353
themes Regression RMSE test: 0.9026817483499576
```

```
['MaxPlayers', 'MfgAgeRec']
themes Regression R2 train: 0.09601348501173823
themes Regression RMSE train: 0.8865346122554537
themes Regression R2 test: 0.09600191502168554
themes Regression RMSE test: 0.918360025157045
```

```
['YearPublished', 'MfgPlaytime', 'MinPlayers']
themes Regression R2 train: 0.05602211128160295
themes Regression RMSE train: 0.9059320636565058
themes Regression R2 test: -0.09553030693563658
themes Regression RMSE test: 1.0109773232967219
```

```
['YearPublished', 'MfgPlaytime', 'MaxPlayers']
themes Regression R2 train: 0.02983135203831888
themes Regression RMSE train: 0.9184136671971993
themes Regression R2 test: -0.15856444013408844
themes Regression RMSE test: 1.0396551598122508
```

```
['YearPublished', 'MfgPlaytime', 'MfgAgeRec']
themes Regression R2 train: 0.11104144189004939
themes Regression RMSE train: 0.8791348122229812
themes Regression R2 test: -0.0033119823743685828
themes Regression RMSE test: 0.9674916277838791
```

['YearPublished', 'MinPlayers', 'MaxPlayers']
themes Regression R^2 train: 0.03594252368509998
themes Regression RMSE train: 0.9155165162467435
themes Regression R^2 test: 0.0381615377421094
themes Regression RMSE test: 0.9472841831375574

['YearPublished', 'MinPlayers', 'MfgAgeRec']
themes Regression R^2 train: 0.129162787718055
themes Regression RMSE train: 0.8701281309846012
themes Regression R^2 test: 0.12976334848983184
themes Regression RMSE test: 0.9010479464934843

['YearPublished', 'MaxPlayers', 'MfgAgeRec']
themes Regression R^2 train: 0.09927629866119925
themes Regression RMSE train: 0.884933254263084
themes Regression R^2 test: 0.09942845857701199
themes Regression RMSE test: 0.9166178819619762

['MfgPlaytime', 'MinPlayers', 'MaxPlayers']
themes Regression R^2 train: 0.05112567449287975
themes Regression RMSE train: 0.9082785708182827
themes Regression R^2 test: -0.09973482298189751
themes Regression RMSE test: 1.0129154713178041

['MfgPlaytime', 'MinPlayers', 'MfgAgeRec']
themes Regression R^2 train: 0.13690397928821574
themes Regression RMSE train: 0.8662520536442256
themes Regression R^2 test: 0.05509388262302284
themes Regression RMSE test: 0.9389090957983317

['MfgPlaytime', 'MaxPlayers', 'MfgAgeRec']
themes Regression R^2 train: 0.10884095862777965
themes Regression RMSE train: 0.880222227367174
themes Regression R^2 test: -0.0029762558284387453
themes Regression RMSE test: 0.9673297440404738

['MinPlayers', 'MaxPlayers', 'MfgAgeRec']
themes Regression R^2 train: 0.12650886763355562
themes Regression RMSE train: 0.8714530019116714
themes Regression R^2 test: 0.12757606482282113
themes Regression RMSE test: 0.9021795987903297

['YearPublished', 'MfgPlaytime', 'MinPlayers', 'MaxPlayers']
themes Regression R^2 train: 0.05646449750168925
themes Regression RMSE train: 0.9057197605763342
themes Regression R^2 test: -0.09398041620378916
themes Regression RMSE test: 1.0102619350703212

```
['YearPublished', 'MfgPlaytime', 'MinPlayers', 'MfgAgeRec']
themes Regression R2 train: 0.14011495207714153
themes Regression RMSE train: 0.8646391950855546
themes Regression R2 test: 0.057770725292347724
themes Regression RMSE test: 0.9375782257721097
```

```
['YearPublished', 'MfgPlaytime', 'MaxPlayers', 'MfgAgeRec']
themes Regression R2 train: 0.11219586157923056
themes Regression RMSE train: 0.878563795592955
themes Regression R2 test: -7.015105420449252e-05
themes Regression RMSE test: 0.9659273175879854
```

```
['YearPublished', 'MinPlayers', 'MaxPlayers', 'MfgAgeRec']
themes Regression R2 train: 0.12964653319300734
themes Regression RMSE train: 0.8698864216208522
themes Regression R2 test: 0.13076293229109825
themes Regression RMSE test: 0.9005303104489946
```

```
['MfgPlaytime', 'MinPlayers', 'MaxPlayers', 'MfgAgeRec']
themes Regression R2 train: 0.13732672631097953
themes Regression RMSE train: 0.8660398812317867
themes Regression R2 test: 0.05626997686062618
themes Regression RMSE test: 0.9383245988966685
```

```
['YearPublished', 'MfgPlaytime', 'MinPlayers', 'MaxPlayers', 'MfgAgeRec']
themes Regression R2 train: 0.14055147198749418
themes Regression RMSE train: 0.864419700550915
themes Regression R2 test: 0.05897818616178285
themes Regression RMSE test: 0.9369772828944758
```

```
[95]: def base_regression(regressor, name):
        regressor.fit(X_train, y_train)

        y_reg_predict=regressor.predict(X_train)

        print(f"{name} Regression R2 train:", r2_score(y_train, y_reg_predict))
        print(f"{name} Regression RMSE train:", root_mean_squared_error(y_train,
        ↪y_reg_predict))

        y_reg_predict = regressor.predict(X_test)

        print(f"{name} Regression R2 test:", r2_score(y_test, y_reg_predict))
        print(f"{name} Regression RMSE test:", root_mean_squared_error(y_test,
        ↪y_reg_predict))
```

```
[96]: from sklearn.linear_model import LinearRegression
```

```
linreg = LinearRegression()  
base_regression(linreg, "Linear")
```

Linear Regression R^2 train: 0.14055147198749418
Linear Regression RMSE train: 0.864419700550915
Linear Regression R^2 test: 0.05897818616178285
Linear Regression RMSE test: 0.9369772828944758

```
[97]: from sklearn.preprocessing import PolynomialFeatures  
from sklearn.pipeline import make_pipeline
```

```
poly_model = make_pipeline(PolynomialFeatures(degree=2), LinearRegression())  
  
base_regression(poly_model, "Polynomial(2)")
```

Polynomial(2) Regression R^2 train: 0.20255821276695007
Polynomial(2) Regression RMSE train: 0.8326533139650326
Polynomial(2) Regression R^2 test: -0.37554943588620415
Polynomial(2) Regression RMSE test: 1.132836719256947

```
[98]: poly_model = make_pipeline(PolynomialFeatures(degree=3), LinearRegression())  
base_regression(poly_model, "Polynomial(2)")
```

Polynomial(2) Regression R^2 train: 0.24850764350465515
Polynomial(2) Regression RMSE train: 0.8083082359604098
Polynomial(2) Regression R^2 test: -27.06100558744434
Polynomial(2) Regression RMSE test: 5.116592514851034

```
[99]: #describe in detail  
from sklearn.linear_model import Ridge, Lasso
```

```
ridge = Ridge(alpha=1.0)  
ridge.fit(X_train, y_train)  
print("Ridge  $R^2$ :", ridge.score(X_test, y_test))  
  
lasso = Lasso(alpha=0.1)  
lasso.fit(X_train, y_train)  
print("Lasso  $R^2$ :", lasso.score(X_test, y_test))
```

Ridge R^2 : 0.05897397991466535
Lasso R^2 : -0.006289727711001447

```
[100]: # Range of alpha values to test (log-spaced for better coverage)  
alphas = np.logspace(-3, 2, 50)
```

```
ridge_scores = []  
lasso_scores = []
```

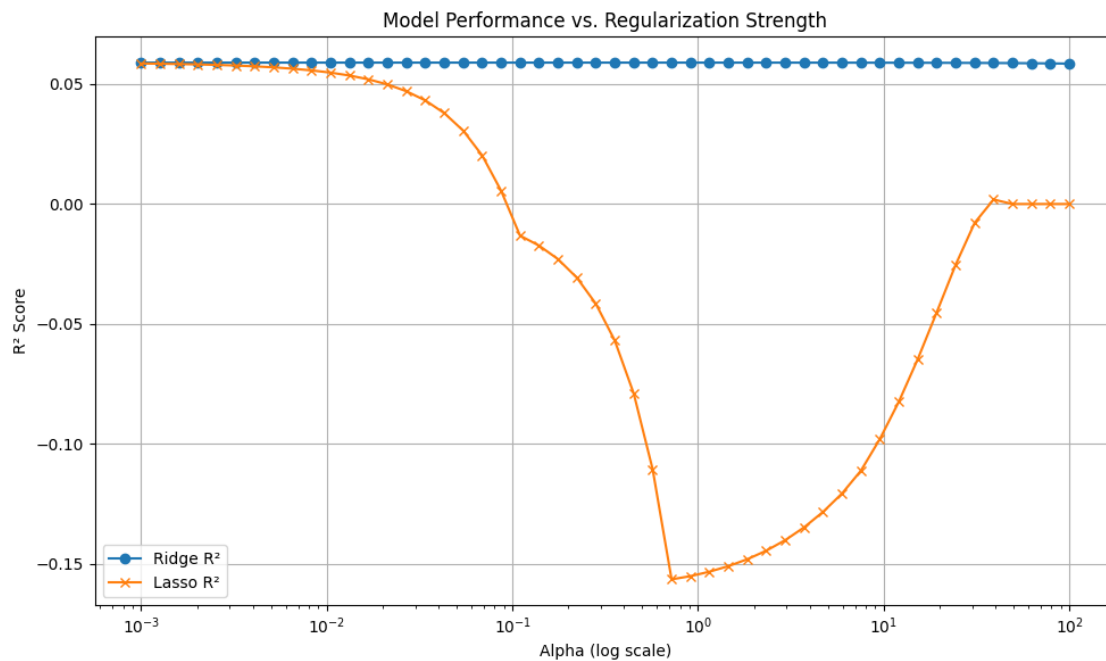
```

for alpha in alphas:
    # Ridge
    ridge = Ridge(alpha=alpha)
    ridge.fit(X_train, y_train)
    ridge_scores.append(ridge.score(X_test, y_test))

    # Lasso
    lasso = Lasso(alpha=alpha, max_iter=10000)
    lasso.fit(X_train, y_train)
    lasso_scores.append(lasso.score(X_test, y_test))

# Plotting
plt.figure(figsize=(10, 6))
plt.plot(alphas, ridge_scores, label='Ridge R2', marker='o')
plt.plot(alphas, lasso_scores, label='Lasso R2', marker='x')
plt.xscale('log')
plt.xlabel('Alpha (log scale)')
plt.ylabel('R2 Score')
plt.title('Model Performance vs. Regularization Strength')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```



```
[101]: from sklearn.ensemble import RandomForestRegressor
forest = RandomForestRegressor(n_estimators=200, max_depth=20, random_state=42)

base_regression(forest, "Forest")
```

Forest Regression R^2 train: 0.7455233013768753
Forest Regression RMSE train: 0.4703690427865899
Forest Regression R^2 test: 0.32387536983660015
Forest Regression RMSE test: 0.7942232464972363

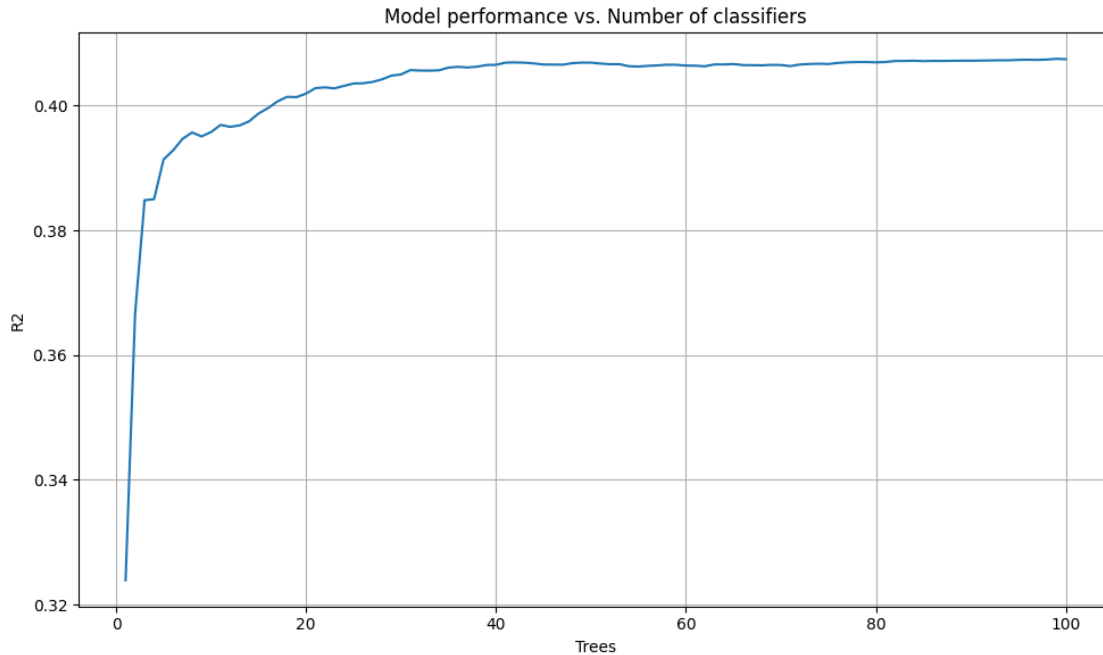
```
[102]: forest = RandomForestRegressor(n_estimators=10, max_depth=10, random_state=42)

base_regression(forest, "Forest")
```

Forest Regression R^2 train: 0.5008905109471664
Forest Regression RMSE train: 0.658737873386394
Forest Regression R^2 test: 0.3957403393694945
Forest Regression RMSE test: 0.7508289633587827

```
[103]: num_trees=np.linspace(1,100,100)
scores=[]
for trees in num_trees:
    forest=RandomForestRegressor(n_estimators=np.int_(trees), max_depth=10,
    random_state=42)
    forest.fit(X_train,y_train)
    y_pred=forest.predict(X_test)
    scores.append(r2_score(y_test,y_pred))

plt.figure(figsize=(10, 6))
plt.plot(num_trees,scores)
plt.xlabel("Trees")
plt.ylabel("R2")
plt.title("Model performance vs. Number of classifiers")
plt.grid()
plt.tight_layout()
plt.show()
```



```
[104]: param_grid = {
        'n_estimators': np.int_(np.linspace(10,100,10)),
        'max_depth': [None, 10, 20, 30],
    }

    reg = RandomForestRegressor(random_state=42)
    grid = GridSearchCV(reg, param_grid, scoring='r2', cv=5, n_jobs=-1)
    grid.fit(X_train, y_train)

    print("Best params:", grid.best_params_)
```

Best params: {'max_depth': 10, 'n_estimators': 80}

```
[105]: base_regression(grid.best_estimator_, "Best forest")
```

Best forest Regression R² train: 0.5122017115374431
 Best forest Regression RMSE train: 0.6512306857288342
 Best forest Regression R² test: 0.40691724205209456
 Best forest Regression RMSE test: 0.7438525655982077

```
[106]: param_grid = {
        'n_estimators': [50, 100, 150, 200],
        'max_depth': [2, 3, 5, 7, 9, 11, 13],
        'min_samples_leaf': [3, 5, 10],
        'min_samples_split': [10, 20, 30],
        'max_features': ['sqrt', 'log2', 0.3]
```

```

}

reg = RandomForestRegressor(random_state=42)
grid = GridSearchCV(reg, param_grid, scoring='r2', cv=5, n_jobs=-1)
grid.fit(X_train, y_train)

print("Best params:", grid.best_params_)

```

Best params: {'max_depth': 11, 'max_features': 'sqrt', 'min_samples_leaf': 3, 'min_samples_split': 30, 'n_estimators': 200}

```
[107]: base_regression(grid.best_estimator_, "Best forest")
```

Best forest Regression R^2 train: 0.47319625351684613
 Best forest Regression RMSE train: 0.6767669590046175
 Best forest Regression R^2 test: 0.4112272660874523
 Best forest Regression RMSE test: 0.7411447914136309

```
[108]: from sklearn.ensemble import GradientBoostingRegressor
```

```

gbr = GradientBoostingRegressor(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=3,
    min_samples_split=10,
    min_samples_leaf=5,
    subsample=0.8,
    random_state=42
)
base_regression(gbr, "Gradient boost")

```

Gradient boost Regression R^2 train: 0.42779963726819437
 Gradient boost Regression RMSE train: 0.7053241991942246
 Gradient boost Regression R^2 test: 0.4062846092114395
 Gradient boost Regression RMSE test: 0.744249188270569

```
[109]: from xgboost import XGBRegressor
```

```

xgb = XGBRegressor(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=3,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42
)
base_regression(xgb, "XGBoost")

```

```

XGBoost Regression R2 train: 0.42708399158643306
XGBoost Regression RMSE train: 0.7057651325835791
XGBoost Regression R2 test: 0.407382147564771
XGBoost Regression RMSE test: 0.7435609629922914

```

```

[110]: X_train=class_df_train
X_test=class_df_test
for data_name in datasets.keys():
    X_train=pd.merge(
        X_train,
        datasets[data_name],
        on='BGGId',
        how='inner'
    )

    X_test=pd.merge(
        X_test,
        datasets[data_name],
        on='BGGId',
        how='inner'
    )

feat_names = X_train.drop(["BGGId", "Name", "NumUserRatings"],axis=1).columns

X_train=scaler.fit_transform(X_train.drop(["BGGId", "Name", "NumUserRatings"],axis=1))
y_train=class_labels_train
X_test=scaler.transform(X_test.drop(["BGGId", "Name", "NumUserRatings"],axis=1))
y_test=class_labels_test

```

```

[111]: xgb = XGBRegressor(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=3,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42
)
base_regression(xgb,"XGBoost")

```

```

XGBoost Regression R2 train: 0.4752282500267029
XGBoost Regression RMSE train: 0.7119165062904358
XGBoost Regression R2 test: 0.4541959762573242
XGBoost Regression RMSE test: 0.7265382409095764

```

```
[112]: linreg = LinearRegression()
base_regression(linreg, "Linear")
```

```
Linear Regression R2 train: 0.6389159877305506
Linear Regression RMSE train: 0.5905381803630868
Linear Regression R2 test: -0.04779512212951453
Linear Regression RMSE test: 1.0066496739641142
```

```
[113]: def xgb_boost_features(features):
    X_train=reg_df_train
    X_test=reg_df_test
    y_train=reg_labels_train
    y_test=reg_labels_test

    for data_name in features:
        X_train=pd.merge(
            X_train,
            datasets[data_name],
            on='BGGIId',
            how='inner'
        )

        X_test=pd.merge(
            X_test,
            datasets[data_name],
            on='BGGIId',
            how='inner'
        )

        X_train=scaler.fit_transform(X_train.drop(["BGGIId", "Name",
↪ "NumUserRatings"],axis=1))

        X_test=scaler.transform(X_test.drop(["BGGIId", "Name",
↪ "NumUserRatings"],axis=1))

    y_train_xg=y_train-1
    y_test_xg=y_test-1

    xgb = XGBRegressor(
        n_estimators=100,
        learning_rate=0.1,
        max_depth=3,
        subsample=0.8,
        colsample_bytree=0.8,
        random_state=42
```

```

)

xgb.fit(X_train, y_train_xg)

print (features)
print("XGBoost train r2:", xgb.score(X_train, y_train_xg))
print("XGBoost test r2:", xgb.score(X_test, y_test_xg))
print('\n')
return xgb

```

```

[114]: import itertools
array=['mechanic', 'subcategory', 'themes', 'publisher']
for L in range(len(array) + 1):
    for subset in itertools.combinations(array, L):
        xgb_boost_features(subset)

```

```

()
XGBoost train r2: 0.4270839930810072
XGBoost test r2: 0.40738215583889614

```

```

('mechanic',)
XGBoost train r2: 0.47396581873181853
XGBoost test r2: 0.461674316273301

```

```

('subcategory',)
XGBoost train r2: 0.4404991882555209
XGBoost test r2: 0.4180297216363237

```

```

('themes',)
XGBoost train r2: 0.4533287984244613
XGBoost test r2: 0.4330331098993606

```

```

('publisher',)
XGBoost train r2: 0.47261037443191767
XGBoost test r2: 0.4492699835946953

```

```

('mechanic', 'subcategory')
XGBoost train r2: 0.47976906073673886
XGBoost test r2: 0.4640708650533578

```

```

('mechanic', 'themes')
XGBoost train r2: 0.483259930996048

```

XGBoost test r2: 0.4666681558807596

('mechanic', 'publisher')
XGBoost train r2: 0.49644934742301416
XGBoost test r2: 0.4811150715095298

('subcategory', 'themes')
XGBoost train r2: 0.4614918493211344
XGBoost test r2: 0.4389538122426283

('subcategory', 'publisher')
XGBoost train r2: 0.48165301882716405
XGBoost test r2: 0.45910780966904197

('themes', 'publisher')
XGBoost train r2: 0.48383713520434823
XGBoost test r2: 0.46181487690256007

('mechanic', 'subcategory', 'themes')
XGBoost train r2: 0.4892760197999302
XGBoost test r2: 0.4695560475399315

('mechanic', 'subcategory', 'publisher')
XGBoost train r2: 0.5041781951108624
XGBoost test r2: 0.48725813170332755

('mechanic', 'themes', 'publisher')
XGBoost train r2: 0.504028339631559
XGBoost test r2: 0.48463827594544606

('subcategory', 'themes', 'publisher')
XGBoost train r2: 0.493265920215884
XGBoost test r2: 0.4708276810465748

('mechanic', 'subcategory', 'themes', 'publisher')
XGBoost train r2: 0.5073682618847579
XGBoost test r2: 0.4896829767589762

```
[115]: array=['mechanic','subcategory','themes', 'publisher', 'artist','designer']
      for L in range(len(array) + 1):
          for subset in itertools.combinations(array, L):
              xgb_boost_features(subset)
```

```
()
XGBoost train r2: 0.4270839930810072
XGBoost test r2: 0.40738215583889614
```

```
('mechanic',)
XGBoost train r2: 0.47396581873181853
XGBoost test r2: 0.461674316273301
```

```
('subcategory',)
XGBoost train r2: 0.4404991882555209
XGBoost test r2: 0.4180297216363237
```

```
('themes',)
XGBoost train r2: 0.4533287984244613
XGBoost test r2: 0.4330331098993606
```

```
('publisher',)
XGBoost train r2: 0.47261037443191767
XGBoost test r2: 0.4492699835946953
```

```
('artist',)
XGBoost train r2: 0.43775312161283597
XGBoost test r2: 0.4162495918153192
```

```
('designer',)
XGBoost train r2: 0.4546708187292433
XGBoost test r2: 0.43779808134991594
```

```
('mechanic', 'subcategory')
XGBoost train r2: 0.47976906073673886
XGBoost test r2: 0.4640708650533578
```

```
('mechanic', 'themes')
XGBoost train r2: 0.483259930996048
XGBoost test r2: 0.4666681558807596
```

('mechanic', 'publisher')
XGBoost train r2: 0.49644934742301416
XGBoost test r2: 0.4811150715095298

('mechanic', 'artist')
XGBoost train r2: 0.47659151348660855
XGBoost test r2: 0.4643736315804855

('mechanic', 'designer')
XGBoost train r2: 0.4861767813223199
XGBoost test r2: 0.47635833107657843

('subcategory', 'themes')
XGBoost train r2: 0.4614918493211344
XGBoost test r2: 0.4389538122426283

('subcategory', 'publisher')
XGBoost train r2: 0.48165301882716405
XGBoost test r2: 0.45910780966904197

('subcategory', 'artist')
XGBoost train r2: 0.44935299709783083
XGBoost test r2: 0.42719270605612936

('subcategory', 'designer')
XGBoost train r2: 0.4664088494509616
XGBoost test r2: 0.4494485433856221

('themes', 'publisher')
XGBoost train r2: 0.48383713520434823
XGBoost test r2: 0.46181487690256007

('themes', 'artist')
XGBoost train r2: 0.45914787394483514
XGBoost test r2: 0.4392718017425189

('themes', 'designer')

XGBoost train r2: 0.4695929697132354
XGBoost test r2: 0.4528395742282517

('publisher', 'artist')
XGBoost train r2: 0.47135466388831193
XGBoost test r2: 0.44692509375901324

('publisher', 'designer')
XGBoost train r2: 0.48330375799703584
XGBoost test r2: 0.462465957553444

('artist', 'designer')
XGBoost train r2: 0.45909710161041006
XGBoost test r2: 0.4427795810705297

('mechanic', 'subcategory', 'themes')
XGBoost train r2: 0.4892760197999302
XGBoost test r2: 0.4695560475399315

('mechanic', 'subcategory', 'publisher')
XGBoost train r2: 0.5041781951108624
XGBoost test r2: 0.48725813170332755

('mechanic', 'subcategory', 'artist')
XGBoost train r2: 0.48411369820535444
XGBoost test r2: 0.4695871297284214

('mechanic', 'subcategory', 'designer')
XGBoost train r2: 0.4942428221705021
XGBoost test r2: 0.4836299411296969

('mechanic', 'themes', 'publisher')
XGBoost train r2: 0.504028339631559
XGBoost test r2: 0.48463827594544606

('mechanic', 'themes', 'artist')
XGBoost train r2: 0.485506826044628
XGBoost test r2: 0.4706982107474661

('mechanic', 'themes', 'designer')
XGBoost train r2: 0.4938834325181809
XGBoost test r2: 0.48007506220362905

('mechanic', 'publisher', 'artist')
XGBoost train r2: 0.49786332326575466
XGBoost test r2: 0.4803136142979645

('mechanic', 'publisher', 'designer')
XGBoost train r2: 0.5033060441073851
XGBoost test r2: 0.4897050638379016

('mechanic', 'artist', 'designer')
XGBoost train r2: 0.48788952416501596
XGBoost test r2: 0.47881546929494845

('subcategory', 'themes', 'publisher')
XGBoost train r2: 0.493265920215884
XGBoost test r2: 0.4708276810465748

('subcategory', 'themes', 'artist')
XGBoost train r2: 0.46737781232039377
XGBoost test r2: 0.4464693977233939

('subcategory', 'themes', 'designer')
XGBoost train r2: 0.4770369360395109
XGBoost test r2: 0.4612504053192866

('subcategory', 'publisher', 'artist')
XGBoost train r2: 0.4847759539261348
XGBoost test r2: 0.46176827954616495

('subcategory', 'publisher', 'designer')
XGBoost train r2: 0.4932793693575136
XGBoost test r2: 0.4721319404563262

('subcategory', 'artist', 'designer')
XGBoost train r2: 0.46834665975664136

XGBoost test r2: 0.45072161036372294

('themes', 'publisher', 'artist')
XGBoost train r2: 0.48311991292068823
XGBoost test r2: 0.4624642859305843

('themes', 'publisher', 'designer')
XGBoost train r2: 0.49127729812523513
XGBoost test r2: 0.4701826198095377

('themes', 'artist', 'designer')
XGBoost train r2: 0.4712335215840512
XGBoost test r2: 0.4533147774181442

('publisher', 'artist', 'designer')
XGBoost train r2: 0.48349277324368845
XGBoost test r2: 0.4625765648662177

('mechanic', 'subcategory', 'themes', 'publisher')
XGBoost train r2: 0.5073682618847579
XGBoost test r2: 0.4896829767589762

('mechanic', 'subcategory', 'themes', 'artist')
XGBoost train r2: 0.491364094834828
XGBoost test r2: 0.4724034556239227

('mechanic', 'subcategory', 'themes', 'designer')
XGBoost train r2: 0.4984101260485293
XGBoost test r2: 0.48491831815023967

('mechanic', 'subcategory', 'publisher', 'artist')
XGBoost train r2: 0.5065042221395359
XGBoost test r2: 0.4909043768180581

('mechanic', 'subcategory', 'publisher', 'designer')
XGBoost train r2: 0.5121940315245654
XGBoost test r2: 0.49825607144856887

('mechanic', 'subcategory', 'artist', 'designer')
XGBoost train r2: 0.49502761203942436
XGBoost test r2: 0.48499902455382327

('mechanic', 'themes', 'publisher', 'artist')
XGBoost train r2: 0.5035499160209503
XGBoost test r2: 0.4853990007368981

('mechanic', 'themes', 'publisher', 'designer')
XGBoost train r2: 0.5082204977497942
XGBoost test r2: 0.49265917255137726

('mechanic', 'themes', 'artist', 'designer')
XGBoost train r2: 0.49557656118188953
XGBoost test r2: 0.48139785609950425

('mechanic', 'publisher', 'artist', 'designer')
XGBoost train r2: 0.5031006908094734
XGBoost test r2: 0.48852313701248273

('subcategory', 'themes', 'publisher', 'artist')
XGBoost train r2: 0.4942099743576962
XGBoost test r2: 0.470098870220089

('subcategory', 'themes', 'publisher', 'designer')
XGBoost train r2: 0.49877006663437584
XGBoost test r2: 0.4783117456717376

('subcategory', 'themes', 'artist', 'designer')
XGBoost train r2: 0.48055943033295423
XGBoost test r2: 0.4612200415837594

('subcategory', 'publisher', 'artist', 'designer')
XGBoost train r2: 0.49343494975031355
XGBoost test r2: 0.47211597711089814

('themes', 'publisher', 'artist', 'designer')
XGBoost train r2: 0.49139064555860446
XGBoost test r2: 0.47110262108896195

('mechanic', 'subcategory', 'themes', 'publisher', 'artist')
XGBoost train r2: 0.5100643135538652
XGBoost test r2: 0.49062525723893213

('mechanic', 'subcategory', 'themes', 'publisher', 'designer')
XGBoost train r2: 0.5136910810425716
XGBoost test r2: 0.4958745484535828

('mechanic', 'subcategory', 'themes', 'artist', 'designer')
XGBoost train r2: 0.500101261909772
XGBoost test r2: 0.48494920778409534

('mechanic', 'subcategory', 'publisher', 'artist', 'designer')
XGBoost train r2: 0.5141331844635546
XGBoost test r2: 0.4977600367405819

('mechanic', 'themes', 'publisher', 'artist', 'designer')
XGBoost train r2: 0.5088971516713747
XGBoost test r2: 0.493264662113295

('subcategory', 'themes', 'publisher', 'artist', 'designer')
XGBoost train r2: 0.4991918434266891
XGBoost test r2: 0.4783667892728073

('mechanic', 'subcategory', 'themes', 'publisher', 'artist', 'designer')
XGBoost train r2: 0.51492684528018
XGBoost test r2: 0.49720644806525316